

Sistemas Operativos

Introducción a sistemas operativos

Componentes de un sistema operativo

1. Usuarios

- Representan a los procesos o sesiones de usuario en ejecución.
- Corresponden a las instancias de programas que interactúan con el sistema.
- Varias instancias pueden funcionar simultáneamente y solicitar servicios al SO.

1. Herramientas, aplicaciones y programas

- Herramientas de desarrollo y utilidades: compilador, ensamblador, editor de texto, intérprete, depurador...
- Aplicaciones de usuario: navegadores, hojas de cálculo, sistemas de bases de datos, servidores web, gestores de impresión, demonios de sistema, etc.
- Todas dependen de las llamadas al sistema para acceder a recursos y están construidas sobre el kernel del SO.

3. Sistema Operativo

- Kernel: capa central que gestiona directamente la CPU, la memoria y los dispositivos de E/S.
- Programas del sistema: servicios en segundo plano y utilidades cercanas al kernel que gestionan tareas avanzadas.
- Proporciona interfaces estables que abstraen la complejidad de hardware y garantizan la seguridad y eficiencia en el acceso a los recursos.

4. Hardware

- Componentes físicos: CPU, RAM, disco/SSD, tarjetas de red, periféricos de E/S.
- Constituye la base sobre la que opera el kernel; todas las demás capas dependen de estos dispositivos.
- El kernel oculta los detalles eléctricos y de diseño de los dispositivos, ofreciendo una visión uniforme y simplificada al resto del sistema.

Módulos de un SO

Dentro del nivel de kernel, el sistema operativo se organiza en varios módulos o subsistemas, cada uno encargado de gestionar un aspecto fundamental de la máquina. Los más importantes son:

1. Sistema de ficheros

- **Función principal:** Proporcionar una estructura lógica para almacenar, organizar y recuperar datos en dispositivos de almacenamiento.
- **Responsabilidades:**
 - **Estructura de directorios:** crear, renombrar y eliminar ficheros y carpetas.

- **Asignación de espacio:** determinar dónde y cómo se guardan los bloques de datos en el dispositivo físico.
- **Control de acceso:** aplicar permisos y listas de control para proteger la integridad y confidencialidad de los datos.
- **Caché de ficheros:** mantener en memoria bloques de disco recientemente accedidos para mejorar el rendimiento de lectura/escritura.

2. Gestión de memoria

- **Función principal:** Controlar el uso de la memoria principal (RAM) garantizando que cada proceso disponga del espacio que necesita de forma aislada y eficiente.
- **Responsabilidades:**
 - **Asignación de memoria:** repartir bloques de memoria a procesos al crearlos o ceder sus datos.
 - **Protección y aislamiento:** evitar que un proceso acceda a la memoria de otro, protegiendo la estabilidad y la seguridad del sistema.
 - **Memoria virtual:** mapear direcciones lógicas a direcciones físicas mediante paginación o segmentación, posibilitando el uso de más memoria que la físicamente disponible.
 - **Swapping o intercambio:** mover páginas de memoria entre RAM y disco cuando la memoria física escasea.

3. Gestión de procesos

- **Función principal:** Crear, programar y coordinar la ejecución de los procesos e hilos que conforman las aplicaciones y servicios del sistema.
- **Responsabilidades:**
 - **Creación y terminación:** invocar `fork()`, `exec()` o sus equivalentes para iniciar nuevos procesos, y liberar recursos al finalizar.
 - **Planificador (scheduler):** decidir qué proceso se ejecuta en cada instante, cuándo ceder la CPU y cómo repartir el tiempo entre procesos.
 - **Sincronización y comunicación:** proporcionar mecanismos para coordinar y compartir datos entre procesos.
 - **Estados de proceso:** gestionar las transiciones entre estados según eventos.

4. Gestión de entrada/salida

- **Función principal:** Facilitar el intercambio de datos entre el computador y sus periféricos.
- **Responsabilidades:**
 - **Drivers:** módulos específicos que conocen el protocolo de cada dispositivo físico y traducen solicitudes genéricas del SO a comandos de hardware.
 - **Colas de E/S y buffering:** almacenar temporalmente datos en búferes o colas para compensar diferencias de velocidad entre CPU/memoria y dispositivos.
 - **Modo bloqueante y no bloqueante:** ofrecer interfaces para que las aplicaciones puedan elegir entre esperar a que termine la operación o continuar con otras tareas.

- **Notificaciones e interrupciones:** utilizar interrupciones hardware para informar al kernel de la finalización de una operación de E/S y despertar procesos bloqueados.

Jerarquía de memoria

En un sistema informático moderno existen diversos tipos de almacenamiento, organizados en una jerarquía según dos criterios fundamentales:

- **Capacidad de almacenamiento:** la cantidad de datos que puede contener cada tipo de memoria.
- **Tiempo de asociamiento:** la rapidez con la que se puede acceder a los datos almacenados.

1. Almacenamiento primario

Esta categoría incluye los tipos de memoria más rápidos que permiten acceder casi instantáneamente a los datos, pero cuya capacidad suele ser muy limitada y su contenido se pierde al apagar el equipo, es **volátil**

- **Registros**
 - Se encuentran dentro de la CPU y tienen acceso instantáneo y se usan para almacenar los datos e instrucciones que el procesador está utilizando en ese preciso momento.
 - Capacidad extremadamente reducida.
- **Memoria caché**
 - También está muy cerca del procesador, y puede estar dividida en tres niveles (L1, L2 y L3).
 - Almacena copias de datos frecuentemente usados para acelerar el acceso.
 - Es más grande que los registros, pero aún muy limitada.
 - Tiempo de acceso muy bajo, aunque mayor que el de los registros.
- **Memoria principal, RAM**
 - Es la memoria general de trabajo del sistema.
 - Se tarda más en acceder que la caché pero su capacidad es mayor.
 - Es volátil.
 - Usada para almacenar temporalmente los programas y datos en ejecución.

2. Almacenamiento secundario

Este nivel incluye medios que permiten almacenar datos de forma **persistente**, pero son más lentos que la memoria principal.

- **Memoria no volátil**
 - Incluye memorias como Flash, EEPROM o ROM.
 - Son rápidas comparadas con discos duros, pero más lentas que la RAM.
 - Se usan en BIOS, almacenamiento USB, SSD, etc.
- **Discos duros y unidades de estado sólido (HDD y SSD)**
 - Alta capacidad de almacenamiento
 - Persistentes

3. Almacenamiento terciario

Incluye tecnologías diseñadas para **almacenamiento masivo a largo plazo**, generalmente en tareas de archivos o respaldo.

- **Discos ópticos (CD, DVD, Blu-ray)**
 - Medios de almacenamiento portátiles y duraderos.
 - Acceso más lento que los discos duros.
 - Capacidad limitada comparada con los SSD o HDD.
- **Cintas magnéticas**
 - Usadas principalmente para copias de seguridad y almacenamiento a muy largo plazo.
 - Extremadamente lentas en acceso, pero con capacidades muy altas.
 - Bajo coste por unidad de almacenamiento.

Relación entre capacidad y velocidad

Existe una relación inversa típica:

- **A mayor velocidad, menor capacidad:** los registros y la caché son muy rápidos, pero su tamaño es mínimo.
- **A mayor capacidad, menor velocidad:** discos, cintas y almacenamiento terciario son muy grandes, pero acceden a datos mucho más lentamente.

Volatilidad frente a persistencia

- **Memorias volátiles:** pierden su contenido al apagar el sistema. Se utilizan para tareas temporales y rápidas.
- **Memorias no volátiles:** conservan los datos sin necesidad de energía. Se utilizan para almacenamiento permanente.

Sistema de ficheros

En los sistemas operativos, los **ficheros** son unidades de almacenamiento de información. Pueden contener datos de usuario, como textos o imágenes, o datos del sistema, como configuraciones o programas. Para poder gestionar estos ficheros de manera eficiente, el sistema operativo utiliza una estructura organizada en varios niveles: **directorios, bloques de datos y bloques de índices**.

1. Directorios

Un **directorio** es una tabla o estructura que asocia nombres de ficheros con su información de localización. Cuando se crea un archivo, el sistema guarda su nombre en el directorio junto con una referencia que indica dónde se encuentra la información del archivo en el almacenamiento. Esta referencia no apunta directamente a los datos, sino a un **bloque de índice**.

2. Bloques de Índice

Un **bloque de índice** es una estructura intermedia que almacena las direcciones de los **bloques de datos** reales dónde está guardado el contenido del archivo. Este bloque actúa como una lista de punteros. Es decir, en lugar de que el directorio conozca directamente los

bloques que forman el archivo, delega esta tarea a un bloque de índice. Esto facilita el manejo de archivos fragmentados y mejora la organización del sistema de archivos.

3. Bloques de Datos

Los **bloques de datos** son unidades físicas de almacenamiento en el disco. Cada bloque puede almacenar una parte del contenido de un archivo grande, por ejemplo, se divide en varios bloques, que no necesariamente están ubicados de forma contigua. El bloque de índice mantiene la relación entre el archivo y los bloques que lo componen.

Gestión de memoria

Memoria virtual

La **memoria virtual** es una técnica que permite que los programas se ejecuten como si dispusieran de una gran cantidad de memoria, aunque la **memoria física** disponible sea limitada. Esta memoria virtual combina **RAM** con **espacio en disco** para ampliar artificialmente la cantidad de memoria utilizable por los procesos.

El sistema operativo se encarga de gestionar esta memoria, dividiendo tanto la memoria física como la memoria virtual en unidades más pequeñas llamadas **páginas**. Gracias a esta organización, los programas pueden cargarse y ejecutarse aunque no estén completamente en memoria física al mismo tiempo.

Swapping

El **swapping** es un mecanismo dentro de la memoria virtual que permite mover procesos o partes de procesos entre la **RAM** y el **almacenamiento secundario** según las necesidades del sistema.

Funcionamiento del Swapping:

1. Carga inicial:

- Supongamos que hay varios programas en ejecución.
- Estos programas están parcialmente cargados en la memoria principal, ocupando distintos bloques de memoria.

2. Memoria llena:

- Cuando la RAM se llena y se necesita cargar otro proceso, el sistema operativo no puede cargarlo directamente si no hay espacio.

3. Swap Out:

- El sistema operativo identifica un proceso inactivo o menos prioritario y **lo mueve temporalmente al disco**, a un espacio llamado **área de swap**.
- Esto libera espacio en la RAM para otros programas más activos.

4. Swap In

- Si el **programa B** vuelve a ser necesario, **se trae de nuevo desde el disco a la RAM**
- No necesariamente se carga en los mismos bloques que ocupaba antes.

Ventajas de la Memoria Virtual y el Swapping

- **Multiprogramación:** Permite ejecutar varios programas al mismo tiempo, incluso si no caben todos completamente en la RAM.

- **Aislamiento:** Cada proceso cree que tiene su propia memoria privada, lo que mejora la seguridad y la estabilidad del sistema.
- **Gestión eficiente:** El sistema operativo puede decidir qué partes de los programas mantener en RAM y cuáles mover al disco, optimizando el rendimiento general.

Multiprogramación

La **multiprogramación** es una técnica fundamental por los sistemas operativos para mejorar la eficiencia del uso del procesador y los dispositivos de entrada/salida.

¿Por qué es necesaria?

Un sólo usuario o programa no puede mantener ocupado al CPU y a los dispositivos de E/S todo el tiempo. Por ejemplo, cuando un programa necesita esperar para leer datos del disco o del teclado, el procesador queda inactivo si no hay otro trabajo disponible.

¿Cómo funciona?

- El sistema operativo mantiene **varios programas** en memoria al mismo tiempo.
- **Sólo uno de ellos se ejecuta activamente en la CPU**, mientras los demás esperan su turno o están en espera por operaciones de E/S.
- Cuando el trabajo actual necesita esperar, el sistema operativo **cambia a otro trabajo disponible**.
- Este mecanismo es controlado por **algoritmos de planificación de trabajos**.

De esta forma, **la CPU siempre tiene algo que hacer**, aprovechando al máximo los recursos del sistema.

Time Sharing

La **multiprogramación** evolucionó hacia el concepto de **time sharing**, o multitarea interactiva, especialmente útil para entornos con **usuarios múltiples**.

Características principales:

- La CPU **cambia rápidamente de un proceso a otro**, dando la ilusión de que todos se ejecutan simultáneamente.
- Esto permite que **varios usuarios interactúen con sus programas en tiempo real**, mientras están ejecutándose.
- El **tiempo de respuesta** esperado debe ser inferior a un segundo para mantener la sensación de interactividad.

Detalles técnicos

- Cada usuario tiene al menos **un proceso en ejecución en memoria**.
- Si hay **varios procesos listos para ejecutarse**, el sistema utiliza **planificación de CPU** para decidir cuál ejecutar a continuación.
- Si los procesos no caben todos en la memoria principal, se utiliza **swapping** para moverlos entre la memoria y el disco.
- Además, la técnica de **memoria virtual** permite ejecutar procesos aunque no estén completamente cargados en RAM.

System Calls

Los sistemas operativos proporcionan una interfaz entre el hardware y los programas de usuario. Esta interfaz se materializa en forma de **System calls**, que permiten a los programas acceder a servicios esenciales como gestión de archivos, procesos, comunicación, entre otros.

Interrupciones y modos de operación

Las llamadas al sistema implican una transición del **modo usuario al modo kernel**. Esta transición puede dar dos tipos de interrupciones

Característica	Interrupción por Hardware	Interrupción por Software
Origen	Dispositivo externo	Instrucción del programa
Generación	Señal eléctrica del hardware	Instrucción específica
Sincronización	Asíncrona	Síncrona
Contador de Programa	No se incrementa	Se incrementa normalmente
Prioridad	Generalmente baja	Alta en relación al código normal
Ejemplo	Pulsación de una tecla	Llamada al sistema para leer un archivo

Modo dual de operación

Para proteger al sistema operativo y a los demás procesos:

- El **hardware** implementa un **bit de modo** que distingue si se está ejecutando en **modo usuario** o **kernel**.
- Algunas instrucciones son **privilegiadas** y sólo se pueden ejecutar en modo kernel.
- Una **llamada al sistema cambia temporalmente al modo kernel**.
- Al retornar al programa de usuario, el sistema vuelve al modo usuario.

Evitar abuso de CPU: uso de temporizador

Para evitar que un proceso consuma el CPU indefinidamente:

- Se configura un **temporizador** antes de ejecutar el proceso.
- Cuando el contador llega a cero, **se genera una interrupción**.
- El sistema operativo puede entonces **interrumpir el proceso**, reasignar la CPU o incluso terminar el proceso si ha superado su tiempo asignado.

Ficheros

Métodos de asignación

En los sistemas de ficheros, uno de los aspectos clave es la **gestión de espacio en disco duro** utilizado por los archivos. Existen distintos **métodos de asignación** de bloques que

permiten almacenar y acceder a los datos de manera eficiente. A continuación, se presentan los métodos más comunes:

1. Contiguous Allocation

En este método, **cada archivo ocupa un conjunto de bloques contiguos** en el disco.

- **Ventajas:**
 - Ofrece el **mejor rendimiento** en la mayoría de los casos, ya que permite el acceso secuencial y directo eficiente.
 - Es un método **simple**: sólo se necesita conocer el número de bloque inicial y la longitud del archivo en bloques.
- **Desventajas:**
 - Requiere **conocer el tamaño de archivo de antemano**.
 - Dificultad para encontrar espacio contiguo para archivos grandes o crecientes.
 - Pueden necesitar **compartir** el disco, tanto **en línea** como **fuera de línea**.
 - Genera **fragmentación externa e interna**

2. Linked Allocation

Con la asignación enlazada, **cada archivo es una lista enlazada de bloques** dispersos por el disco.

- Cada bloque contiene **un puntero al siguiente** bloque del archivo.
- **Ventajas:**
 - No hay fragmentación externa.
 - Ideal para archivos de tamaño dinámico
- **Desventajas:**
 - No permite **acceso directo** ni aleatorio, ya que para llegar a un bloque específico es necesario recorrer los anteriores.
 - Mayor **sobrecarga** por el almacenamiento de punteros.
 - Puede requerir compactación.

3. File Allocation Table (FAT)

Es una variación del método enlazado que mejora el acceso:

- Una **tabla en el inicio del volumen** almacena los punteros de todos los bloques del disco.
- La tabla es **indexada por número de bloque**
- Se puede **cargar en memoria principal**, lo que mejora el rendimiento al permitir acceso directo a través de la tabla.
- **Ventajas:**
 - Evita la lectura secuencial de punteros en disco.
 - Asignar nuevos bloques es sencillo.

4. Index Allocation

Este método asigna a cada archivo uno o más **bloques índice**, que contienen **punteros directos** a los bloques de datos del archivo.

- Permite **acceso directo y dinámico** sin fragmentación externa.
- Cada archivo tiene su propia **tabla de índice**.
- **Ventajas:**
 - Soporta archivos de tamaño variable, ideal para operaciones que requieren acceso aleatorio.
- **Desventajas:**
 - Hay una **sobrecarga por los bloques de índice**.
 - Requiere mayor gestión para archivos grandes.

Sistema de ficheros UNIX

Introducción al sistema de archivos: visión lógica.

La estructura lógica típica de un sistema de archivos UNIX incluye:

- **BOOT:** contiene el gestor de arranque necesario para iniciar el sistema operativo.
- **SUPERBLOCK:** Mantiene información sobre el sistema de archivos, como tamaño, número de bloques, número de inodos, etc.
- **INODE LIST:** Estructura que almacena metadatos de los archivos.
- **DATA AREA:** Zona en la que se guarda el contenido real de los archivos.

Comandos relacionados

- `du` - Muestra el uso del disco por archivos y directorios.
- `df` - Muestra el uso del espacio en disco por sistemas de archivos.
- `mount / umount` - Monta y desmonta sistemas de archivos.
- `mkfs` - Crea un nuevo sistema de archivos.

Montaje de sistemas de archivos

El comando `mount` permite unir dos sistemas de archivos en uno solo. Por ejemplo, al insertar un dispositivo, este se puede montar dentro del sistema raíz con: `mount /dev/fd0 /mnt`

Ejemplos comunes:

- Montar un segundo disco: `mount -t ext4 /dev/hda1/ home2`
- Montar una unidad de CD: `mount -r -t iso9660 /dev/scd0 /cdrom`
- Montar una memoria USB: `mount -w -o noatime /dev/sda1 /memstick`
- Desmontar la USB: `umount /dev/sda1` o `umount /memstick`

Comandos útiles:

- `lsblk` - Muestra los dispositivos de bloques conectados.
- `blkid` - Muestra información sobre los dispositivos de bloques.
- `cfdisk`, `parted` - Herramientas de particionado.

Opción para mount	Descripción
auto / noauto	Montaje automático al inicio o solo bajo petición del usuario (<code>mount -a</code>).
user / nouser	Permite (o no) que usuarios sin privilegios monten el dispositivo.

Opción para mount	Descripción
exec / noexec	Permite (o no) ejecutar archivos binarios en el sistema de archivos.
ro	Monta el sistema en modo de solo lectura.
rw	Monta el sistema en modo lectura-escritura.
sync / async	Las operaciones de escritura se realizan de forma síncrona (seguras). Las escrituras se hacen de forma asíncrona (más rápidas pero arriesgadas).
default	Usa las opciones por defecto: <code>rw</code> , <code>suid</code> , <code>dev</code> , <code>exec</code> , <code>auto</code> , <code>nouser</code> , <code>async</code> .

Estructura típica de directorios en UNIX

Directorio	Descripción
/	Raíz del sistema de archivos.
/bin	Ejecutables básicos necesarios por todos los usuarios.
/sbin	Herramientas administrativas del sistema (<code>fsck</code> , <code>mount</code> , etc.).
/boot	Archivos necesarios para el arranque del sistema.
/dev	Representación en archivos de los dispositivos del sistema.
/etc	Archivos de configuración del sistema.
/home	Directorios personales de los usuarios.
/lib	Bibliotecas esenciales compartidas y módulos del kernel.
/media	Punto de montaje automático para dispositivos extraíbles.
/mnt	Punto de montaje manual para dispositivos o sistemas remotos.
/proc	Sistema de archivos virtual que representa procesos activos.
/root	Directorio personal del superusuario (root).
/tmp	Archivos temporales. Puede vaciarse al reiniciar.
/usr	Programas, bibliotecas y recursos no críticos del sistema.
/usr/bin	Ejecutables no esenciales (como <code>gcc</code> , <code>gedit</code> , etc.).
/usr/include	Cabeceras de desarrollo (<code>stdio.h</code> , <code>math.h</code> ...).
/usr/lib	Bibliotecas para programas en <code>/usr</code> y otros.
/var	Archivos variables como logs, colas de impresión y correo.
/var/log	Archivos de registro del sistema.
/var/email	Correos entrantes de los usuarios.
/var/spool	Colas de tareas (correo, impresión...).
/var/tmp	Archivos temporales que se conservan entre reinicios.

Bibliotecas estáticas y dinámicas

En UNIX, las bibliotecas de desarrollo están en **/usr/lib** y se clasifican en:

- **Bibliotecas estáticas:** *.a
- **Bibliotecas dinámicas:** *.so

Ejemplo en C:

```
#include <stdio.h>
#include <math.h>
int main()
{
    float x = 0.5, y;
    y = sin(x);
    printf("sin(%f) = %f\n", x, y);
    return 0;
}
```

- **Enlace estático:** `gcc ejemplo.c -l,`

Se enlaza con `/usr/lib/libm.a`, incluyendo el código de `sin()` directamente en el ejecutable. Resultado: binario más pesado.

- **Enlace dinámico:** `gcc ejemplo.c -lm`

Usa `/usr/lib/libm.so`. El código de `sin()` se carga en tiempo de ejecución. Ventaja: menos espacio, y las bibliotecas pueden actualizarse sin recompilar.

Visión física de las particiones del disco

Particiones del disco en un PC

En los sistemas UNIX y similares, el disco duro se divide físicamente en **particiones**, que permiten organizar el almacenamiento en secciones independientes. Cada partición puede contener un sistema de archivos diferente, y una de ellas puede estar dedicada al arranque del sistema.

MBR (Master Boot Record) por dispositivo de almacenamiento: El **MBR**, o Registro de Arranque Maestro, es una pequeña sección al principio de un dispositivo de almacenamiento, cuya función es contener el cargador de arranque y la tabla de particiones principal. El MBR ocupa los **primeros 512 bytes** del disco y está presente en **cada dispositivo de almacenamiento**, no por partición.

Sector de arranque por partición: Cada partición que contenga un sistema operativo debe tener su **sector de arranque**. Este sector incluye código específico, como el cargador de arranque del sistema, que se encarga de iniciar la carga del sistema operativo.

Registro de arranque de partición extendida: Cuando el número de particiones primarias no es suficiente, se puede crear una **partición extendida**, dentro de la cual se pueden definir múltiples **particiones lógicas**. Cada partición lógica contiene su propio **registro de arranque**

de partición extendida, permitiendo así gestionar más particiones dentro de un esquema limitado.

Estructura del Master Boot Record

El MBR está compuesto por:

1. Código de arranque principal: pequeño programa que localiza el sector de arranque de una partición activa y transfiere el control a él.
2. **Tabla de particiones**: contiene información sobre hasta cuatro particiones primarias.
3. **Firma de arranque**: valor fijo 0x55AA que indica un MBR válido.

Offset	Tamaño	Descripción
0x000	446 bytes	reservado
0x1BE	16 bytes	Entrada partición 1
0x1CE	16 bytes	Entrada partición 2
0x1DE	16 bytes	Entrada partición 3
0x1EE	16 bytes	Entrada partición 4
0x1FE	2 bytes	0xAA55

Limitaciones del esquema de particionado MBR

El esquema MBR, creado a principios de los años 80, presenta **limitaciones técnicas** importantes:

- Sólo permite **cuatro particiones primarias**.
- Máximo tamaño por partición: **2 TB**.
- No es compatible con discos modernos con tamaños superiores ni con algunos métodos avanzados de arranque como UEFI

GPT: La evolución del particionado

Para superar estas limitaciones, **Intel desarrolló en los años 90** un nuevo formato de tabla de particiones: el **GPT (GUID Partition Table)**. Actualmente, **la mayoría de los sistemas operativos modernos lo soportan** y es el esquema estándar en sistemas con firmware UEFI. Las ventajas de GPT incluyen:

- Permite un **número casi ilimitado de particiones**.
- Soporta discos de **más de 2TB**.
- Mayor fiabilidad y redundancia gracias a la presencia de **copias de respaldo** de la tabla de particiones.

System V vs BSD

Los sistemas UNIX han utilizado diferentes estructuras lógicas de sistema de archivos a lo largo del tiempo. Dos de las más influyentes son el **System V File System** y el **BSD Fast File System**. A continuación se detalla su organización lógica y las diferencias clave entre ambos.

Estructura lógica en System V

El sistema de archivos de **System V** organiza el disco en cuatro zonas principales.

1. **BOOT**: contiene el código de arranque necesario para iniciar el sistema operativo desde esa partición.
2. **SUPERBLOQUE**: almacena información general sobre el sistema de archivos, como el tamaño total, el número de inodos, bloques libres, etc.
3. **Lista de inodos**: colección de estructuras que describen los archivos.
4. **Área de datos**: espacio del disco donde se almacena el contenido real de los archivos y directorios.

Estructura lógica en BSD

El **BSD Fast File System** mejora la organización del disco dividiéndolo en **grupos de cilindros**, lo cual favorece el rendimiento y reduce la fragmentación. Cada grupo de cilindros contiene sus propias copias de estructuras clave, lo que mejora la fiabilidad y el acceso local a los datos.

La organización es la siguiente:

1. **Boot**: código de arranque.
2. **Superbloque**: bloque principal con la información del sistema de archivos.
3. **Grupos de cilindros**:
 - CG 0
 - CG 1
 - ...
 - CG N

Cada grupo de cilindros contiene:

- **superbloque (replicado)**: copia del superbloque para mejorar la tolerancia a fallos.
- **Cabecera del grupo de cilindros**: información de control sobre el grupo.
- **Lista de inodos del grupo i**: describe los archivos asignados a ese grupo.
- **Área de datos del grupo i**: donde se almacenan los datos reales de los archivos.

Organización del disco en grupos de cilindros.

La división en grupos de cilindros permite:

- **Adaptar inodos y datos cercanos físicamente** para mejorar el rendimiento.
- Reducir el **tiempo de búsqueda** del cabezal del disco.
- Tener **copias de seguridad parciales** de estructuras críticas, como el superbloque.

Bloques y fragmentos en BSD

El sistema BSD utiliza una estrategia eficiente para el uso del espacio en disco mediante **bloques grandes y fragmentos**. Esta técnica permite reducir el desperdicio de espacio, especialmente en archivos pequeños.

Funcionamiento:

- Los archivos se almacenan en bloques grandes.

- El **último bloque** de archivo, si no está completamente lleno, se almacena como un **fragmento** más pequeño.

Esta técnica ayuda a **minimizar la fragmentación interna** y aprovechar mejor el espacio disponible en disco.

Inodos

El sistema operativo asocia un **inodo** a cada archivo.

Esta estructura contiene toda la información necesaria para gestionar el archivo, salvo su nombre, que se almacena en el directorio.

Es importante **diferenciar** entre:

- **Inodos en disco:** ubicados en la *Lista de Inodos*, forman parte permanente del sistema de archivos.
- **Inodos en memoria:** almacenados en la *Tabla de Inodos*, se gestionan dinámicamente y su estructura es similar a la de la *Buffer cache*.

Estructura de los Inodos

Cada inodo en disco contiene la siguiente información sobre el archivo:

- **Propietario:** identificador del usuario dueño del archivo.
- **Grupo:** grupo de usuarios asociado al archivo.
- **Tipo de archivo:** indica si es un archivo regular, directorio, enlace simbólico, etc.
- **Permisos de acceso:** lectura, escritura y ejecución para propietario, grupo y otros.
- **Fechas del archivo**
 - Último acceso.
 - Última modificación del contenido
 - Última modificación del propio inodo
- **Número de enlaces:** número de hard links que apuntan a este inodo.
- **Tamaño del archivo:** en bytes
- **Direcciones de disco:** punteros a los bloques del disco donde se almacenan los datos reales del archivo.
- **Direcciones de disco:** punteros a los bloques del disco donde se almacenan los datos reales del archivo

Tipos de archivo y permisos de archivo

Los **tipos de archivo** y los **permisos de acceso** son gestionados mediante el *modo del archivo*, una parte del inodo.

Comandos relacionados:

- `chmod` : cambia los permisos de acceso de un archivo o directorio. También existe la *syscall* `chmod()` .
- `chown` : cambia el propietario de un archivo o directorio. Existe también como *syscall* `chown()` .

Cambiar los permisos con chmod

Hay **dos formas de sintaxis**:

1. **Modo simbólico**: `chmod u+rw g-x o+x archivo.txt`

- u = usuario, g = grupo, o = otros
-
- añade permisos, - los elimina

2. **Modo simplificado para todos los usuarios**: `chmod a+rx archivo.txt`

- a = todos
- r = lectura, w = escritura, x = ejecución.

"Sticky bit"

Significado tradicional en archivos ejecutables:

- Indicaba que el **código del programa** se mantenía en memoria tras su ejecución para acelerar futuros usos.
- Actualmente, **el kernel de Linux ignora el sticky bit en archivos**.

Significado actual en directorios:

Se usa para **proteger archivos en directorios públicos** como /tmp.

Es un directorio con sticky bit: **Sólo el superusuario, el dueño del directorio o el dueño del archivo** pueden borrar o renombrar entradas dentro del directorio.

Fechas de los archivos

Cada archivo en UNIX almacena **tres marcas de tiempo**, accesibles mediante la estructura stat:

- `st_atime` : Fecha del último acceso.
- `st_mtime` : Fecha de la última modificación de datos.
- `st_ctime` : Fecha de última modificación del inodo.

Links

Conceptos clave:

- **Hard links**:
 - Dos o más entradas de directorio que **apuntan al mismo inodo**.
 - Son **idénticos al archivo original**: comparten contenido, permisos y metadatos del inodo.
 - **No se pueden crear hard links a directorios ni entre sistemas de archivos diferentes**
- **Symbolic link**:
 - Es un **archivo independiente** que contiene la **ruta** hacia otro archivo o directorio.
 - Puede apuntar a **archivos o directorios**, incluso si están en otro sistema de archivos o partición.

Nota importante

A diferencia de los hard links, **los enlaces simbólicos pueden apuntar a archivos o directorios en otros sistemas de archivos o particiones.**

Estructura del diseño por bloques en el disco

Cada archivo en el sistema de archivos UNIX está asociado a:

- Un **inodo** en la **lista de inodos**-
- Una serie de **bloques de datos** en el área de datos del disco.

Bloques de datos del archivo

El inodo contiene **direcciones de disco** que permiten acceder a los bloques que contienen el contenido del archivo. El acceso a estos bloques se organiza jerárquicamente para permitir el crecimiento eficiente del archivo, aunque **el acceso se vuelve más lento a medida que el archivo crece**, debido a los niveles de indirección.

Esquema de direcciones de bloque

Suponiendo:

- Tamaño del bloque **1 Kbyte** (1024 bytes)
- Tamaño de una dirección: **4 bytes**

Un inodo típico incluye:

- **10 direcciones directas** -> Acceso directo a bloques de datos.
- **1 dirección de bloque indirecto simple** -> Apunta a un bloque que contiene 256 direcciones directas a bloques de datos.
- **1 dirección de bloque indirecto doble** -> Apunta a un bloque que contiene 256 direcciones a otros bloques, que a su vez contienen 256 direcciones a bloques de datos.
- **1 dirección de bloque indirecto triple** -> Apunta a un bloque que contiene 256 direcciones a bloques, que contienen 256 direcciones a otros bloques, que finalmente contienen 256 direcciones a bloques de datos.

Cálculo del tamaño máximo de un archivo

1. Direcciones directas (10 bloques)

Cada dirección apunta directamente a un bloque de datos.

- **Fórmula:**
- **Capacidad:**

2. Bloque indirecto simple

Un bloque de punteros que almacena direcciones.

- **Fórmula:**
- **Capacidad total:**

3. Bloque indirecto doble

Apunta a 256 bloques indirectos simples.

- **Fórmula:**
- **Capacidad total:**

4. Bloque indirecto triple

Apunta a 256 bloques indirectos dobles.

- **Fórmula:**
- **Capacidad total:**

Directorios

Un **directorio** es un archivo cuyo contenido se interpreta como una **lista de entradas de directorio**. Estas entradas asocian un **nombre de archivo** con el **número de inodo** que lo representa en el sistema de archivos.

Formato de las entradas de directorio.

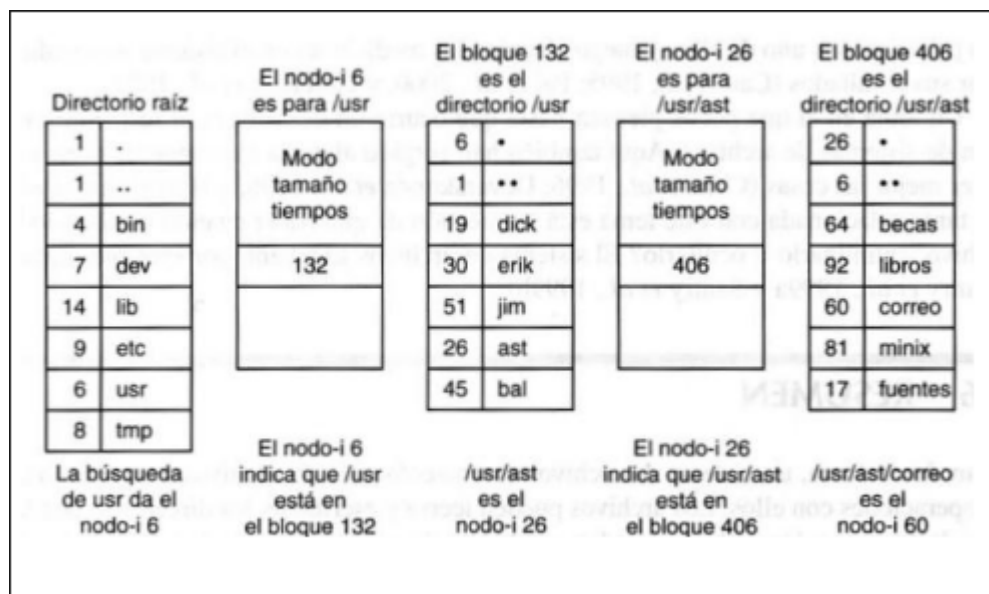
Los formatos pueden variar según la implementación del sistema operativo:

- **Formato System V:**
 - **Número de inodo:** 2 bytes
 - **Nombre del archivo:** 14 bytes
- **Formato BSD:**
 - **Número de inodo:** 4 bytes
 - **Longitud de la entrada:** 2 bytes
 - **Longitud del nombre del archivo:** 2 bytes
 - **Nombre del archivo:** cadena terminada en \0, completada hasta múltiplo de 4 bytes

Las funciones estándar para manipular directorios están definidas en `<dirent.h>`:

- `opendir()` - Abre un directorio para su lectura.
- `readdir()` - Lee una entrada del directorio
- `closedir()` - Cierra el descriptor del directorio.

Cálculo del número mínimo de accesos al disco



En el caso **óptimo**:

- **Bloques leídos del área de datos:**
 - Bloque del directorio /
 - Bloque 132 (de /usr)

- Bloque 406 (de /usr/ast)

Total **3 accesos**.

- **Accesos a inodos en la lista de inodos del disco:**

- Se acceden a los inodos: 1, 6, 26 y 60

Total: **4 inodos**

Pero, cuántos bloques de disco se leyeron realmente?

- Si asumimos que los 4 inodos están almacenados en un **mismo bloque de inodos**, entonces:
 - Sólo se necesita **una lectura de bloque** de la lista de inodos.

Estructuras del kernel: sistema de archivos

El kernel del sistema operativo utiliza diversas estructuras para gestionar el acceso a los archivos y optimizar el rendimiento del sistema.

Una de las más importantes es el **Buffer Caché**, que regula el flujo de datos entre los dispositivos de almacenamiento secundario y el núcleo. Su función principal es **reducir el número de accesos físicos al disco**, almacenando temporalmente bloques de datos en memoria. De forma similar, existe un mecanismo asociado a la **memoria virtual**, conocido como **Page Cache**, que actúa como una caché para páginas de memoria, incluyendo aquellas relacionadas con archivos mapeados en memoria o accedidos frecuentemente.

Esquema de principales estructuras del kernel del sistema de archivos

Las estructuras clave del kernel en la gestión del sistema de archivos incluyen:

- **Superbloque:** Contiene información general del sistema de archivos, como su tamaño, número de inodos, y localización de bloques de datos.
- **Lista de inodos en memoria:** Conjunto de inodos cargados desde disco a memoria. Cada inodo representa un archivo o directorio, con metadatos como permisos, tamaño y punteros a bloques de datos.
- **Tabla de archivos abiertos por el sistema:** Contiene una entrada por cada archivo abierto en el sistema, con punteros a su inodo y posición actual de lectura / escritura.
- **Tabla de descriptores de archivo por proceso:** Cada proceso mantiene su propia tabla de descriptores que apuntan a las entradas de la tabla global de archivos abiertos.
- **Buffer Caché:** Área de memoria que almacena bloques de datos recientemente leídos o escritos, para evitar accesos repetidos al disco.
- **Page Cache:** Memoria caché de páginas virtuales, muchas de las cuales corresponden a datos de archivos usados por los procesos.

Llamadas al sistema para el sistema de archivos

El sistema operativo proporciona un conjunto de **llamadas al sistema** para permitir que los procesos puedan interactuar con los archivos. Estas funciones están definidas en bibliotecas del sistema como `<fcntl.h>` y `<unistd.h>`, y permiten operaciones como abrir, leer, escribir o duplicar descriptores de archivo.

```
int open(char *nombre, int modo, int permisos);
```

Abre un archivo y devuelve un **descriptores de archivo**, que será un número entero que representa el archivo abierto.

- Nombre: ruta del archivo
- Modo: modo de apertura
- Permisos: permisos (utilizados con O_CREAT para indicar los permisos iniciales del archivo).

Modos posibles

Pueden combinarse con operadores OR |

- 0: sólo lectura
- 1: sólo escritura
- 2: lectura y escritura

O usando constantes de `<fcntl.h>`

- `O_RDONLY` : sólo lectura.
- `O_WRONLY` : sólo escritura.
- `O_RDWR` : lectura y escritura.
- `O_APPEND` : añadir al final.
- `O_CREAT` : crear archivo si no existe.

```
int read(int df, char *buff, int n);
```

Lee datos del archivo referenciado por el descriptor `df` y los almacena en el búfer `buff`.

- `df` : descriptor del archivo abierto.
- `buff` : dirección en el espacio del usuario donde se guardan los datos.
- `n` : número de bytes a leer

```
int write(int df, char *buff, int n);
```

Escribe en el archivo referenciado por `df` los datos contenidos en el búfer `buff`.

- `df` : descriptor del archivo abierto.
- `buff` : dirección de los datos a escribir.
- `n` : número de bytes a escribir.

Duplicación de descriptores

```
int dup(int df);
```

Crea un **nuevo descriptor de archivo** que referencia al **mismo archivo abierto** que el descriptor `df`. Devuelve el **primer descriptor libre disponible**.

```
int dup2(int df, int newfd);
```

Hace que `newfd` sea una copia exacta de `df`. Si `newfd` ya estaba en uso, se cierra automáticamente antes de ser reutilizado.

Ejecutables con SETUID

El kernel de UNIX asocia **dos identificadores de usuario** a cada proceso:

1. **UID real**: identifica al usuario que ha lanzado el proceso.

2. **UID efectivo**: se utiliza para verificar permisos de acceso a archivos, asignar la propiedad de los archivos recién creados y controlar el envío de señales a otros procesos.

El kernel permite que un proceso cambie su UID efectivo al ejecutar un programa con **bit SETUID activado**, o al invocar explícitamente la llamada al sistema `setuid()`

¿Qué es un programa SETUID?

Es un ejecutable que tiene activado el bit SETUID en su campo de permisos. Cuando se ejecuta, el kernel **establece al UID efectivo como el propietario del archivo ejecutable**, en lugar del UID real del proceso.

Notas sobre cifrado y autenticación

Además del clásico **DES**, hoy en día se utilizan algoritmos más avanzados como **AES (Advanced Encryption Standard)** con tamaños de clave de 128, 192 o 256 bits.

La mayoría de sistemas Linux emplean **funciones hash** para la autenticación:

- MD5: produce un hash de 128 bits.
- SHA-1: produce un hash de 160 bits.

Llamada al sistema `setuid()`

```
int setuid(uid_t uid);
```

Esta llamada cambia el UID efectivo del proceso y su comportamiento depende del UID efectivo actual.

Casos en los que tiene éxito

1. **Si el proceso es ejecutado por el superusuario**

El kernel cambia tanto el UID real como el efectivo al valor pasado como parámetro.

2. **Si el proceso *no* es ejecutado por el superusuario:**

- Si uid coincide con el UID real del proceso -> éxito
- Si uid coincide con el UID efectivo previamente guardado -> éxito.
- En cualquier otro caso -> error.

Sistema de archivos Linux Ext2fs

El sistema de archivos **Ext2fs** (Second Extended File System) utiliza una estrategia similar a la del sistema de archivos BSD Fast File System (**FFS**) para localizar los bloques de datos asociados a un archivo determinado.

Pero, existen diferencias significativas entre ambos, especialmente en sus políticas de asignación de disco.

Comparación con FFS:

- En **FFS**, los archivos se almacenan en bloques de 8 KB, los cuales pueden dividirse en **fragmentos de 1 KB**. Esto aprovecha mejor el espacio en disco al manejar archivos pequeños o bloques parcialmente llenos al final de un archivo.
- En contraste, **Ext2fs no utiliza fragmentos**. En su lugar, realiza la asignación directamente en bloques de tamaño menor.

Tamaños de bloque en Ext2fs:

- El tamaño por defecto de bloque en **Ext2fs** es de **1 KB**, aunque se admiten bloques de **2 KB** y **4 KB** dependiendo de la configuración al formatear el sistema de archivos.

Políticas de asignación de bloques:

- **Ext2fs** emplea políticas de asignación diseñadas para colocar bloques lógicamente contiguos de un archivo en bloques **físicamente contiguos en disco**.
- Esta estrategia mejora el rendimiento, ya que permite al sistema enviar **una única petición de E/S** para leer o escribir múltiples bloques, reduciendo así el número de accesos al disco.

Sistema de archivos con Journaling

Los sistemas operativos mantienen un *Buffer caché* de los datos y metadatos de los archivos para optimizar el acceso al sistema de archivos.

Problema: Cuando ocurre un **apagón eléctrico** o un **fallo del sistema**, puede haber **inconsistencias** en el sistema de archivos si los datos modificados en la caché aún no se han escrito en disco.

Comando relacionado: `fsck` (file system check). Herramienta que verifica y repara inconsistencias en sistemas de archivos no transaccionales.

Journaling

Un **sistema de archivos con journaling** es un sistema **resiliente a fallos**, que garantiza la integridad de los datos. La clave de su funcionamiento es:

- Antes de actualizar los bloques originales en disco, los **metadatos** se escriben primero en un **registro secuencial**.
- Sólo cuando esta operación se ha completado, se procede a escribir los datos en disco.
- En caso de fallo, el código de recuperación analiza el journal y **reproduce las operaciones pendientes**, resultando así la coherencia del sistema sin necesidad de verificar todo el disco.

Sistema de archivos con journaling en Linux:

- ext3
- ext4
- ReiserFS
- XFS
- JFS

Ventaja de journaling

Un sistema con journaling no necesita revisar todo el sistema de archivos. Sólo los **inodos y directorios** implicados en la operación fallida, mejorando así la velocidad y eficiencia de la recuperación.

Memoria

Introducción

La **memoria** es un componente esencial en cualquier ordenador, ya que permite almacenar y recuperar información. A nivel de hardware, está compuesto por circuitos electrónicos, y su unidad básica de almacenamiento es el **bit**. Sin embargo, las computadoras pueden trabajar con **bytes**, y en muchos sistemas, se utilizan unidades mayores llamadas **palabras**.

- Una **palabra** es la unidad natural de datos utilizada por un procesador. La mayoría de los registros del procesador tienen tamaño de palabra y las operaciones de transferencia entre la memoria y la CPU suelen realizarse en unidades de palabra.
- En sistemas modernos, el tamaño típico de palabra es de **32 o 64 bits**.

Tiempos de acceso a memoria

La memoria se puede clasificar según la forma en que se accede a la información:

- **Memoria convencional:** Se accede directamente a través de una dirección. Se le proporciona una dirección de memoria y devuelve el contenido almacenado.
- **Memoria Asociativa o CAM:** Se accede a través de un contenido. El sistema busca si ese contenido está almacenado en algún lugar. Si lo encuentra, devuelve las direcciones correspondientes. Se utiliza en cachés y funciona de forma similar a una tabla hash.

Jerarquía de memoria

El **tiempo de acceso a memoria** es el tiempo necesario para leer o escribir datos en la memoria, desde que se coloca la dirección hasta que la CPU accede o almacena los datos. Existe un compromiso entre velocidad, capacidad y coste.

- Memorias más rápidas son más caras.
- Memorias con mayor capacidad suelen ser más lentas.

Esto lleva al diseño de una **jerarquía de memoria**, donde cada nivel tiene diferentes características de velocidad y capacidad:

1. **Registros del procesador**
2. **Memoria caché**
3. **Memoria principal**
4. **Disco duro**
5. **Medios más lentos**

Fragmentación de memoria

La fragmentación de memoria puede ocurrir en dos formas:

- **Fragmentación interna:** Ocurre cuando se asignan bloques de tamaño fijo, pero el proceso no necesita todo ese bloque, dejando espacio desperdiciado dentro del mismo bloque.
- **Fragmentación Externa:** Aparece cuando existen suficientes bloques libres en total, pero no están contiguos, por lo que no se pueden usar para satisfacer una solicitud de gran tamaño. Es común en sistemas de segmentación pura.

El papel del sistema operativo en la gestión de la memoria

El **sistema operativo** es el gestor de recursos y uno de los recursos más importantes que administra la memoria:

- Lleva el **registro** de la memoria: cuánto está libre y cuánto está asignado.
- Tiene políticas de **asignación de memoria**.
- Asigna memoria a los procesos cuando la solicitan.
- Libera la memoria cuando los procesos terminan.
- Administra el **sistema de memoria virtual**, permitiendo que los procesos usen más memoria de la físicamente disponible.

Espacio de direcciones virtuales del proceso

Cada proceso tiene un **espacio de direcciones virtual** dividido en regiones según su función:

- **Texto o código:** contiene el código ejecutable del programa.
- **Datos estáticos:** variables globales o estáticas con o sin inicialización.
- **Heap:** memoria dinámica asignada durante la ejecución.
- **Stack:** contiene los *frames* de pila con variables locales y direcciones de retorno de funciones.

Espacio de direcciones

¿Qué es el espacio de direcciones de un proceso?

El espacio de direcciones de un proceso es el conjunto de direcciones de memoria que dicho proceso puede acceder. Es un límite impuesto tanto por el hardware como por el sistema operativo para proteger y organizar el uso de la memoria.

¿Qué ocurre si un proceso accede a una dirección fuera de su espacio de direcciones?

No puede. Esa es, precisamente, la definición del espacio de direcciones: es el conjunto de direcciones a las que el proceso puede acceder.

¿Y qué ocurre si un proceso *intenta* acceder a una dirección fuera de su espacio de direcciones?

El hardware detecta esta acción ilegal, notifica al sistema operativo, y este normalmente termina el proceso con un error del tipo "*segmentation fault*".

¿Qué es el espacio de direcciones de usuario de un proceso?

Es el subconjunto del espacio de direcciones al que el proceso puede acceder cuando se encuentra en modo usuario (user mode), es decir, cuando no tiene privilegios de sistema.

¿Por qué el espacio de direcciones de un proceso no coincide con la memoria física de la máquina?

Por varias razones:

- En una misma máquina pueden ejecutarse múltiples procesos, cada uno con su espacio de direcciones.
- En los sistemas modernos, las direcciones que ve un proceso son lógicas o virtuales, no físicas.
- El espacio de direcciones puede ser mayor que la memoria física instalada.

- La estructura del espacio de direcciones permite la portabilidad del programa entre distintas máquinas.
- Algunas direcciones dentro del espacio de direcciones pueden estar reservadas para fines especiales.

¿Cómo está estructurado el espacio de direcciones de un proceso?

Está dividido en zonas (también llamadas segmentos o regiones), cada una con una función específica. Las zonas típicas incluyen:

- **Segmento de código:** instrucciones del programa.
- **Segmento de datos:** variables globales y estáticas.
- **Pila (stack):** para variables locales y llamadas a funciones.
- **Región para memoria dinámica (heap).**
- **Regiones mapeadas:** para archivos o memoria compartida.

¿Puede crecer el espacio de direcciones de un proceso?

Sí, puede crecer de varias formas:

- Al solicitar más memoria dinámica (por ejemplo, `malloc`), el segmento de datos o *heap* puede expandirse.
- Al mapear archivos, se crean nuevas regiones para el contenido del archivo.
- Al realizar llamadas recursivas o funciones con muchas variables locales, la pila puede crecer.
- Al usar memoria compartida, se crean nuevas regiones asignadas a esta memoria.

¿Puede reducirse el espacio de direcciones de un proceso?

Sí. Cuando un proceso libera memoria (por ejemplo, con `free` o al cerrar un *mapeo*), el espacio de direcciones puede disminuir y algunas regiones pueden liberarse completamente.

¿Existe algún comando para ver el espacio de direcciones de un proceso?

Sí. Algunos comandos disponibles en distintos sistemas operativos son:

- `pmap` en Linux y Solaris.
- `procstat -vm` en FreeBSD.
- `procmap` en OpenBSD.
- `vmmap` en macOS.

¿El espacio de direcciones de un proceso debe ser contiguo?

No. Puede tener "huecos" (holes). De hecho, esto es necesario para permitir el crecimiento

de algunas regiones (por ejemplo, la pila o el *heap*) sin interferir con otras regiones. Estos huecos también ayudan a mantener una separación segura entre zonas de memoria.

Swap

¿Qué es el área de swap?

El área de **swap** es una parte de almacenamiento secundario utilizada como memoria auxiliar por el sistema operativo. Su función principal es permitir que el sistema soporte un mayor nivel de multiprogramación, es decir, más procesos en ejecución simultáneamente, aunque no todos quepan en la memoria principal.

¿Cuándo se utiliza el swap?

Un proceso en ejecución necesita estar en memoria. Si la RAM está llena, el sistema operativo puede **mover temporalmente procesos o partes de procesos al área de swap** para liberar espacio. Esto permite ejecutar otros procesos que sí necesitan estar en RAM en ese momento. Sin embargo, si un proceso que está en la zona de swap es seleccionado por el planificador, debe **ser cargado de nuevo en memoria**, lo que incrementa el tiempo de cambio de contexto. En el caso de procesos que están esperando operaciones de entrada/salida, el sistema primero transfiere los datos a **buffers del núcleo**, y luego a los dispositivos de E/S, añadiendo más sobrecarga al sistema.

Swapping vs Paging

Swapping

- Consiste en mover **procesos completos** al área de swap.
- Se utilizaba en sistemas antiguos y ahora obsoletos.
- Permitía aumentar la multiprogramación con un alto coste de rendimiento.

Paging

- Se utiliza en los sistemas modernos con **memoria virtual**.
- En lugar de mover procesos completos, se transfieren **páginas individuales** del proceso.
- Ofrece una **ilusión de memoria casi ilimitada**, al gestionar de forma eficiente las necesidades de memoria de los procesos.

En resumen, hoy en día **los sistemas operativos modernos rara vez hacen swapping completo**, y en su lugar usan **paging**, que es mucho más eficiente.

Tipos de dispositivos de swap

El área de swap puede estar implementada como:

- Un **disco dedicado exclusivamente** al swap.
- Una **partición** específica dentro de un disco.
- Un **archivo dentro del sistema de archivos**.

Swap como archivo

- Es una **solución más flexible**: se puede cambiar fácilmente su tamaño o ubicación.
- Pero es **menos eficiente**, ya que implica acceder al swap a través del sistema de archivos, lo que añade capas de indirección.

Swap en diferentes sistemas operativos

- **MS Windows** utiliza un archivo de swap.
- **Sistemas tipo Unix** suelen usar **particiones de swap**, aunque también pueden configurarse para usar archivos si se desea más flexibilidad.

Reubicación y protección

¿Cuándo y dónde se asignan las direcciones de memoria?

1. Relocalización absoluta

- Las direcciones se determinan en tiempo de compilación y/o enlace.
- En ejecutable **sólo puede ejecutarse** en una posición específica de memoria.
- **No hay portabilidad**: no puede ejecutarse en otras ubicaciones.

2. Relocalización estática

- Las direcciones se calculan al **cargar** el programa en memoria.
- El ejecutable contiene **referencias relativas**.
- El procesador **no puede ser movido** a otra posición una vez cargado.
- El swapping sólo es posible si el proceso regresa a la misma ubicación.

3. Relocalización dinámica

- Las direcciones se determinan en **tiempo de ejecución**.
- El proceso trabaja con **direcciones lógicas o virtuales**, no físicas.
- Traducción realizada por **hardware especializado**.
- Permite **mayor flexibilidad**: los procesos pueden moverse libremente por la memoria.
- Permite **enlace dinámico**: bibliotecas compartidas.
- Requiere traducción de direcciones lógicas a físicas, **los sistemas modernos usan relocalización dinámica**

Protección de la memoria

La protección de memoria es esencial para garantizar la **estabilidad y seguridad** del sistema:

Reglas básicas de protección

- Un proceso **no debe acceder** directamente a la memoria del sistema operativo.
- Un proceso **no debe acceder** a la memoria de otros procesos.

Soporte hardware básico: dos registros de límites

- **Registro base**: dirección física mínima accesible por el proceso.
- **Registro límite**: tamaño del espacio lógico que puede utilizar el proceso.

Funcionamiento

- Cada dirección lógica generada por el proceso:
 - Debe ser **menor que el valor del registro límite**.
Luego se **suma al valor del registro base** para obtener la dirección física.
- Si la dirección lógica excede el límite, el hardware genera una **excepción**.

- Los valores de estos registros se actualizan en cada **cambio de contexto** y se almacenan en el **PCB** (Process Control Block).
- Cambiar estos valores es una **instrucción privilegiada**

Ventajas

- **Aislamiento de procesos.**
- El programa tiene la ilusión de ejecutarse en su propia máquina, con una memoria que comienza en 0.
- Este esquema corresponde a un **sistema segmentado con un único segmento.**

Protección moderna

- **Segmentación y paging** son las técnicas actuales que permiten:
 - Protección efectiva de la memoria.
 - Relocalización dinámica.
- Es necesario contar con al menos **dos modos de ejecución**:
 - **Modo usuario**: ejecución restringida.
 - **Modo kernel**: acceso completo, reservado al sistema operativo.

Esquemas simples de gestión de memoria

Sistemas sin multiplicación

Este enfoque ha estado **obsoleto durante décadas**. En estos sistemas:

- Sólo existían **dos zonas de memoria**:
 - Una para el **sistema operativo**.
 - Otra para el **proceso de usuario**.
- Disposición típica:
 - El SO en las posiciones **bajas de memoria**.
 - El resto asignado al proceso de usuario.

Sistemas con multiprogramación

Para los Sistemas Operativos multiprogramados, el esquema más simple consiste en **dividir la memoria en particiones**, cada una conteniendo un proceso. Hay 2 alternativas:

1. Partición de tamaño fijo
 - **Número fijo de procesos** en memoria.
 - Problemas:
 - **Fragmentación interna**
 - **Fragmentación externa**
2. Particiones de tamaño variable
 - **Tamaño y número de particiones pueden cambiar** dinámicamente.
 - Ventajas:
 - **Fragmentación interna despreciable**
 - Inconvenientes:
 - **Fragmentación externa** sigue presente.

- Se requería **compactación de memoria**, una operación **muy costosa**.

Segmentación (Obsoleta)

La **segmentación** fue una técnica de gestión de memoria que **reflejaba la forma en que los programadores estructuran los programas**: como una colección de segmentos lógicos.

- **Segmento**: unidad lógica como: programa principal, funciones, métodos, objetos, variables locales/globales, pila, tablas de inodos, arreglos, etc.
- Cada proceso tiene un **espacio de direcciones dividido en segmentos** de tamaño variable.

Direcciones en segmentación

Una **dirección lógica** se expresa como un par: <número de segmento, desplazamiento>.

El número de segmento apunta a una **entrada en la tabla de segmentos**:

- **Base**: dirección física inicial del segmento.
- **Límite**: tamaño del segmento.

Hardware y protección

- El registro **Segment Table Base Register, STBR**, señala la ubicación de la tabla de segmentos.
- Cambiar registros de segmento en un cambio de contexto es una **instrucción privilegiada**.
- La protección se implementa en cada entrada de la tabla:
 - **Bit de validación**
 - **Permisos**
 - **Modo de acceso**
- **Compartir código o datos a nivel de segmento**.

Fragmentación

- **Fragmentación interna**: Tamaños de segmento múltiplos de una unidad fija. Espacio desaprovechado **dentro del segmento**. En general, **despreciable**.
- **Fragmentación externa**: Los segmentos son de tamaño variable -> se crean **huecos de tamaños diversos** -> **Compactación**, resuelve el problema, pero con alto coste computacional.

Asignación dinámica del almacenamiento

- El SO gestiona bloques libres y ocupados. Al liberar memoria, **fusiona huecos adyacentes**.

Algoritmos de asignación:

- **First fit**: primer hueco que quepa. Rápido
- **Next fit**: desde el último hueco usado.
- **Best fit**: hueco más pequeño que sea suficiente.
- **Worst fit**: hueco más grande. Suele ser menos eficiente.

Se ha demostrado que *first fit* y *best fit* tienen mejor rendimiento que *worst fit*.

Estimación de memoria desaprovechada:

- Sean:
 - : tamaño promedio de segmento.
 - : tamaño promedio de hueco.
- Total de memoria desaprovechada respecto al total:

Soporte en hardware

- Requiere soporte de hardware para:
 - Traducción dinámica de direcciones.
 - Protección de memoria.
 - Compartición de segmentos.
- No requiere que el espacio de direcciones lógico y físico tengan el mismo tamaño.

Paginación

La paginación es una técnica de gestión de memoria que permite que el espacio de direcciones físicas de un proceso sea no contiguo. Es decir, un proceso puede ser cargado en cualquier lugar disponible de la memoria física. Esto soluciona varios problemas importantes:

- **Envita la fragmentación externa**, aunque introduce fragmentación interna.
- **Elimina la necesidad de dividir la memoria en bloques de tamaño variable.**
- **Permite la reubicación dinámica, protección de memoria y compartición entre procesos.**

Estructura de paginación

- **Memoria física:** se divide en bloques de tamaño fijo llamados *frames*, cuyo tamaño es potencia de 2, típicamente entre 512 bytes y 16 MB.
- **Memoria lógica (virtual):** se divide en bloques del mismo tamaño llamados *pages*.
- Los espacios de direcciones lógicas y físicas no necesitan tener el mismo tamaño.

El sistema operativo mantiene una lista de *frames* libres. Para ejecutar un proceso que necesita N páginas, se buscan N *frames* libres donde cargar el programa. Para traducir direcciones lógicas a físicas, el sistema operativo configura una **page table**, cuya dirección base se almacena en un registro especial llamado **Page Table Base Register**. Este registro se actualiza en cada cambio de contexto, ya que es una instrucción privilegiada.

Traducción de Direcciones

La dirección generada por la CPU se divide en dos partes:

- **Número de página:** índice en la tabla de páginas.
- **Desplazamiento:** posición dentro de la página.

La entrada correspondiente en la tabla de páginas da el número de *frame* físico donde se encuentra la página lógica. La dirección física final es la combinación del *frame* base y el desplazamiento.

Características de Paging

- **No hay fragmentación externa.**
- **Fragmentación interna** crece con el tamaño de página.
- **Páginas pequeñas:** menor desperdicio, pero mayor sobrecarga por tablas de páginas grandes.
- El sistema operativo debe gestionar tanto los *frames* libres como las tablas de páginas activas.
- Cada proceso ve su propia memoria lógicas, totalmente distinta de la física.

Protección y Compartición

- Un proceso sólo puede acceder a su propia memoria a través de su tabla de páginas.
- La tabla sólo puede ser modificada por el sistema operativo.
- Es posible compartir memoria: dos procesos pueden tener entradas en sus tablas apuntando al mismo *frame* físico.

Soporte de Hardware

- Algunos procesadores antiguos no soportan paginación.
- Se requiere hardware de traducción, como **TLB (Translation Lookaside Buffer)**.

TLB: Memoria asociativa para traducción rápida

- Caché rápida que almacena pares <número de página lógicas, número de *frame* físico>.
- Si hay acierto, se accede directamente al *frame* físico.
- Si hay fallo, se consulta la tabla de memoria y se actualiza la TLB.
- Algunas TLBs usan identificadores de espacio de direcciones para evitar limpiar la TLB en cada cambio de contexto.

Tablas de paginación invertidas

- En sistemas con espacio de direcciones muy grandes, las tablas de páginas pueden ser enormes.
- Soluciones:
 - **Paginación multinivel**
 - **Tablas de páginas invertidas**

En una **tabla de páginas invertida**, cada entrada corresponde a un ***frame* físico**, no a una página lógica. Contiene información del proceso y número de página que ocupa ese *frame*. Se usa una función hash para acelerar búsquedas. Utilizado en arquitecturas como **IMB PowerPC** y **AS/400**.

Sistemas mixtos

Los sistemas modernos de gestión de memoria utilizan combinaciones de técnicas de paginación y/o segmentación. Aunque existen variantes, la paginación siempre está como último mecanismo de gestión, ya que permite un uso más eficiente de la memoria.

Tipos de sistemas mixtos

- **Segmented Paging:** Obsoleto hoy en día. Fue utilizado en sistemas como el **IBM System/370**, donde la tabla de páginas de un proceso estaba segmentada.
- **Paged segmentation:** Variante donde los segmentos de memoria están paginados.
- **Multilevel Paging:** Se implementan varias capas de tablas de páginas para reducir el uso de memoria y optimizar el acceso.

Los sistemas actuales utilizan esquemas más sofisticados:

- **Arquitectura Intel x86 de 32 bits:** Paginación segmentada con **dos niveles** de tablas de páginas.
- **Arquitectura Intel x86 de 64 bits:** Utiliza cuatro niveles de paginación.

Ventajas de la paginación multinivel

A priori, la paginación multinivel es más compleja y puede ser más lenta que la paginación de un sólo nivel, aunque esto depende del rendimiento de la memoria, las cachés y la TLB.

Arquitecturas Intel de 32 y 64 bits

Arquitectura Intel de 32 bits

En los PC de 32 bits, se utiliza un sistema de **segmentación combinada con dos niveles de paginación**.

- **Segmentación:**
 - Cada dirección lógica se compone de un **selector** y un **desplazamiento**.
 - El selector contiene 13 bits para el índice del segmento, 1 bit para seleccionar la tabla y 2 bits de nivel de privilegio.
 - Se accede a una **tabla de descriptores**, de donde se obtiene una dirección base y un límite
- Cálculo de dirección lineal:
 - La dirección base del segmenteo se suma al desplazamiento, generando una **dirección lineal** de 32 bits.
- **Paginación en dos niveles:**
 - Los 10 bits más significativos de la dirección lineal indican una entrada en la **tabla de directorios de página**.
 - Los siguientes 10 bits se usan para indexar la **tabla de páginas** correspondiente.
 - Los últimos 12 bits indican el **desplazamiento dentro de la página**

Cada tabla de páginas tiene 1024 entradas, por lo que ocupa exactamente **una página**.

Arquitectura Intel de 64 bits

En la práctica, incluso los procesadores de 64 bits no utilizan todas las direcciones posibles. Actualmente se manejan:

- **48 bits de direcciones virtuales**
- **52 bits de direcciones físicas**

Direcciones virtuales válidas

- Los 16 bits más significativos deben ser todo 0s o todo 1s.

- Esto divide el espacio en dos mitades:
 - **Mitad inferior:** 0x0000000000000000 a 0x00007FFFFFFFFFFFFF
 - **Mitad superior:** 0xFFFF800000000000 a 0xFFFFFFFFFFFFFFFF
- Las direcciones intermedias no se consideran válidas.

Paginación de cuatro niveles

- **Tamaño de página:** 4 KB
- **12 bits:** desplazamiento dentro de la página.
- **32 bits:** número de página virtual, dividimos en 4 niveles de 9 bits cada uno
 - Cada nivel tiene 512 entradas, y cada entrada ocupa 8 bytes -> una tabla ocupa 4 KB
- El número de página virtual se consulta en la **TBL (Translation Lookaside Buffer)**
 - Si hay acierto, se obtiene el **número de página física (PPN)** de 40 bits.
- Se suma el desplazamiento -> Dirección física de 52 bits.
- Esta dirección física se usa para acceder a la **caché L1**, con:
 - 40 bits para la etiqueta (tag).
 - 6 bits de índice
 - 6 bits de desplazamiento

Introducción a memoria Virtual

¿Dónde están las páginas?

Esto plantea otra pregunta: **Si una página no está en memoria, ¿dónde está? ¿De dónde la saca el sistema operativo?**

Depende de si la página es de **código, datos o pila (stack)** y de si ha sido **modificada**:

- **Código:** Páginas de solo lectura, se cargan desde el archivo ejecutable.
- **Datos:**
 - Si están **inicializados**, se cargan desde el archivo ejecutable.
 - Si **no tienen valores iniciales**, se asignan nuevas páginas y se **rellenan con ceros** (por eso en C las variables externas y estáticas sin inicializar se ponen a 0).
 - Si fueron **modificadas**, se toman del **dispositivo de intercambio (swap)**.
- **Pila (stack):**
 - Las páginas nuevas no modificadas se **asignan y se llenan con ceros**.
 - Las ya usadas y modificadas se toman del **dispositivo de intercambio**.

Por tanto, para tener **memoria virtual**, se necesita un **dispositivo de almacenamiento auxiliar llamado "dispositivo de intercambio" (swap)** para guardar las páginas modificadas de datos y pila.

¿Qué se necesita para tener memoria virtual?

La implementación más común es la **paginación por demanda**, y para que funcione se requiere:

1. **Un esquema de direccionamiento con paginación y bit de presencia**, para que al referenciar una página con bit 0, la MMU genere una excepción.

2. **Un dispositivo de almacenamiento para guardar páginas**, llamado **dispositivo de intercambio** (swap). Puede ser:
 - **Un archivo** (como en Windows, más flexible).
 - **Una partición de disco** (como en Unix, más rápida al evitar la sobrecarga del sistema de archivos).
3. **Un sistema operativo capaz de gestionar esa excepción**:
 - Guarda el estado del proceso.
 - Si la dirección es ilegal actúa en consecuencia (normalmente termina el proceso).
 - Si es válida pero la página no está en memoria localiza la página en el swap, la carga, actualiza la tabla de páginas y reanuda el proceso.

La memoria virtual no es tan simple

En la explicación anterior, la memoria virtual parece simple, pero hay **muchos detalles adicionales**. Por ejemplo, en la instrucción "leer la página en memoria" hay varias consideraciones:

- ¿**Planificamos otro proceso** mientras se espera al dispositivo?
- ¿**Tenemos un *frame* libre** en memoria?
- Si no hay marco libre, habrá que **reemplazar una página** existente:
 - ¿Debe ser una página del mismo proceso? (esto se llama **ámbito de reemplazo**).
 - ¿Cómo decidir qué página reemplazar? (esto es la **política de limpieza**).
 - ¿Cargamos una sola página o intentamos anticipar y cargar varias? (**política de precarga** o *fetching*).

Segmentos de software

Segmentación vs paginación

Ya hemos visto qué eran la segmentación y la paginación a nivel de hardware. Incluso hemos visto ejemplos reales de segmentación paginada, como en la arquitectura de PC de 32 bits de Intel.

Por un lado, la segmentación se acerca más a la estructura lógica de un programa, ya que permite separar regiones como el código, los datos, el heap, las bibliotecas, la pila, etc.

Por otro lado, la paginación es mucho más eficiente en cuanto a memoria, ya que todas las páginas tienen el mismo tamaño. Cada paginación puede ser reemplazada por otra.

Por eso, la mayoría de los sistemas operativos implementan la memoria virtual mediante **paginación bajo demanda**. Aún así, existieron algunos sistemas operativos que no lo hicieron, como el OS/2 1.x de IBM, que utilizaban segmentación bajo demanda.

En la actualidad, la mayoría del hardware sólo proporciona paginación, y aunque hay hardware que también ofrece segmentación, esta rara vez se utiliza.

¿Qué es un segmento de software?

Pensemos en un ejemplo muy simplificado.

Supongamos un sistema paginado de 16 bits: 6 bits para el número de página y 10 para el desplazamiento dentro de la página.

Ahora consideramos un proceso muy simple que tiene:

- Código en las páginas 0, 1 y 2
- Datos en las páginas 4, 5, 6, 7 y 8
- Pila en las páginas 38 y 39

Traducido a direcciones lógicas, el proceso tendría:

- Código que comienza en la dirección 0x0000, con un tamaño de 3 KB (Hasta 0x0BFF)
- Datos que comienzan en 0x1000, con un tamaño de 5 KB (Hasta 0x23FF)
- Pila que comienza en 0xE000, con un tamaño de 2 KB (Hasta 0xE7FF)

Imaginemos ahora que el sistema operativo mantiene el espacio de direcciones del proceso como una lista de elementos, cada uno describiendo una "zona" distinta del espacio de direcciones.

Cada elemento tiene la dirección virtual de inicio, el tamaño y los permisos.

Fallos de página y errores de segmentación

Imaginemos que todas las páginas del proceso, excepto la página 5 y la página 6, están en memoria. Esto significa que los bits de presencia para las páginas 0, 1, 2, 4, 7, 8, 38 y 39 están en 1, y el resto en 0.

- Si el proceso accede a la dirección 0x124A, que está en la página 4, la MMU la traducirá a una dirección física normalmente.
- Si accede a 0x14F2 (página 5), como su bit de presencia es 0, la MMU generará una **excepción**. El sistema operativo tomará el control y comprobará que esa dirección está dentro del rango 0x1000 a 0x23FF, por tanto es válida. Se trata de un **fallo de página**, y el sistema operativo lo atenderá.
- Si accede a la dirección 0x4F06 (página 19), su bit de presencia también es 0, y la MMU lanzará una excepción. Sin embargo, el sistema operativo comprobará que esa dirección **no pertenece a ningún segmento** definido. Esto genera un **error de segmentación**, lo que normalmente resulta en la finalización del proceso.

Esto demuestra que podemos tener "segmentos" de software aunque no haya segmentación por hardware. Linux llama a estas entidades **segmentos**, mientras que otros sistemas operativos las llaman **regiones**. El comando `pmap` muestra los segmentos de un proceso, incluyendo su dirección de inicio, tamaño y permisos.

Tipos de segmentos

Existen dos tipos principales de segmentos:

1. **Segmentos vnode**: Están asociados a un archivo, como los segmentos de código o archivos mapeados. Las páginas de estos segmentos obtienen sus valores iniciales del archivo al que están asociados.
2. **Segmentos anónimos**: No están asociados a ningún archivo. Ejemplos de estos son los segmentos de datos y pila. Se asocian finalmente al dispositivo de swap. Las páginas de estos segmentos se inicializan con ceros cuando se produce un fallo de página por primera vez.

En Linux y Solaris, el comando `pmap` muestra los segmentos del espacio de direcciones de un proceso, junto con su dirección de inicio y tamaño, y en el caso de segmentos vnode, el archivo asociado.

Copy-on-Write

Existen dos situaciones en las que los segmentos deben duplicarse, por ejemplo durante la llamada al sistema `fork()`.

- Si los segmentos son de sólo lectura, simplemente se **comparten**.
- Si son de escritura, se aplica un procedimiento llamado **Copy-on-Write**:
 - El segmento se comparte pero se marca como de sólo lectura.
 - Si se intenta escribir en ese segmento, se produce una excepción.
 - El sistema operativo toma el control y comprueba que la excepción se ha producido al intentar modificar un segmento originalmente de escritura pero ahora marcado como sólo lectura.
 - Entonces, **duplica la página** que causó la excepción, actualiza la tabla de páginas de uno de los procesos, y el acceso puede continuar.

Este mecanismo evita duplicar todo el segmento; **sólo se duplican las páginas realmente modificadas**

¿Qué es la memoria virtual y por qué funciona?

Memoria Virtual

La **memoria virtual** es una técnica que permite ejecutar programas que no están completamente cargados en la memoria física. Esto es posible porque, en la práctica, no todas las partes de un programa se utilizan durante su ejecución. Por ejemplo, algunos errores raros, funciones opcionales que nunca se llaman o variables sobredimensionadas pueden no llegar a utilizarse.

¿Por qué funciona la memoria virtual? Principio de localidad

El **principio de localidad** es la razón principal por la que la memoria virtual funciona. Este principio establece que, durante la ejecución de un programa, las referencias a memoria tienden a agruparse tanto en el proceso como en el tiempo:

- **Localidad temporal**: direcciones que se han usado recientemente tienden a usarse de nuevo pronto.
- **Localidad espacial**: si se accede a una dirección de memoria, es probable que pronto se acceda a direcciones cercanas.

El sistema operativo puede aprovechar esto asignando memoria sólo a las porciones activamente utilizadas del programa, reduciendo la cantidad de memoria física necesaria.

Ventajas de la memoria virtual

El uso de memoria virtual ofrece varias ventajas importantes:

- Permite ejecutar programas más grandes que la memoria RAM instalada.
- Aumenta el grado de multiprogramación, ya que varios procesos pueden compartir la RAM.

- Reduce el uso de entrada/salida durante el intercambio de programas completos, ya que sólo se intercambian las páginas necesarias.

Implementación: Paginación bajo demanda

La forma más común de implementar memoria virtual es mediante **paginación bajo demanda**. En esta técnica, las principales páginas del programa se cargan a la memoria sólo cuando son necesarias.

Este método permite ejecutar programas que superan el tamaño de la memoria física, a costa de una cierta pérdida de rendimiento. Esto introduce la necesidad de decidir **qué página reemplazar** cuando no hay *frames* libres, y si hacerlo con un algoritmo de reemplazo **local** (dentro del proceso) o **global** (compartido entre procesos).

También es clave definir **cuántos frames se asignan a cada proceso** para optimizar el rendimiento y minimizar fallos de página.

Desempeño de la paginación bajo demanda

El servicio de un fallo de página incluye:

1. **Atención del fallo de página:** el sistema operativo toma el control, guarda el estado del proceso y localiza la página en el dispositivo de intercambio.
2. **Transferencia de la página:** espera en la cola del dispositivo, realiza la búsqueda, espera la latencia, transfiere la página y actualiza la tabla de páginas.
3. **Reanudación del proceso:** se restaura el estado del proceso y se continúa su ejecución.

Thrashing

El **thrashing** ocurre cuando el sistema pasa más tiempo manejando fallos de página que ejecutando procesos útiles. Esto sucede cuando:

- Un proceso no tiene suficientes *frames* asignados para su localidad activa.
- Cada nuevo acceso reemplaza una página que aún es necesaria, generando un ciclo de fallos constantes.

Si se usa reemplazo **global**, este problema puede propagarse a otros procesos, afectando al sistema completo.

Prevención del thrashing

Para evitar el thrashing, el sistema operativo debe asignar suficientes *frames* a cada proceso. Hay dos estrategias principales para decidir esto:

- **Modelo del conjunto de trabajo:** intenta identificar cuántas páginas usa activamente un proceso.
- **Modelo de frecuencia de fallos de página:** ajusta los *frames* asignados en función de la tasa de fallos del proceso.

Ubicación, carga y reemplazo de páginas en memoria virtual

Consideraciones generales

El diseño de un sistema de memoria virtual implica tomar diversas decisiones fundamentales. Cada una de ellas afecta directamente al rendimiento, eficiencia y complejidad del sistema. Entre las principales cuestiones a resolver se encuentran:

- ¿Dónde se coloca en memoria la página recientemente referenciada? -> **Política de ubicación de páginas**
- ¿Sólo se carga la página que causó el fallo? -> **Política de carga de páginas**
- ¿La página que se reemplaza pertenece al mismo proceso que causó el fallo? -> **Ámbito de reemplazo**
- ¿Qué criterio se utiliza para elegir la página a reemplazar? -> **Algoritmo de reemplazo**

Política de ubicación de páginas

La política de ubicación de páginas define dónde se colocará en la memoria física una página cuando es traída desde el almacenamiento secundario.

- En sistemas **paginados**, este problema no es relevante, ya que todos los *frames* son iguales.
- En sistemas **segmentados**, sí es una precaución importante.
- En sistemas de **segmentación paginada**, la memoria también se asigna por páginas, por lo que tampoco suele representar un problema.
- En sistemas **NUMA**, sí es un aspecto crítico debido a las diferencias de latencia entre nodos de memoria.

Política de carga de páginas

Cuando ocurre un fallo de página, el sistema operativo puede optar por diferentes estrategias de carga:

- **Paginación por demanda pura**: sólo se carga la página que fue referenciada. Esto implica que el sistema no carga ninguna página de un proceso hasta que esta sea usada.
- **Prefetching**: Se carga la página requerida y algunas páginas adicionales. Esta técnica aprovecha que muchos dispositivos de swap funcionan más eficientemente al transferir varias páginas de una vez. Si embargo, puede introducir ineficiencias si las páginas adicionales no llegan a utilizarse.

Comapración:

- La paginación por demanda produce muchos fallos inicales, pero con el tiempo disminuyen al cargarse más páginas.
- El prefetching puede reducir fallos, pero con el riesgo de cargar páginas innecesarias.

Reemplazo de páginas

La política de reemplazo determina qué página será descartada de la memoria física cuando sea necesario hacer espacio para una nueva. Este proceso involucra tres aspectos funamentales:

1. **Asignación de *frames***: ¿Cuántos **frames** se asignan a cada proceso?

2. **Ámbito del reemplazo:** ¿Puede una página de un proceso reemplazar a otra de un proceso diferente?
3. **Algoritmo de reemplazo:** ¿Qué criterio se usa para seleccionar la página a eliminar?

Asignación de *frames*

Existen dos enfoques principales:

- **Asignación fija:** a cada proceso se le asigna un número fijo de *frames*.
- **Asignación variable:** el número de *frames* asignados a un proceso varía según sus necesidades durante la ejecución.

En la asignación fija, surge la cuestión de cómo determinar cuántos *frames* asignar:

- Mismo número para todos los procesos.
- Proporcional al tamaño del proceso.
- Según prioridad del proceso, tiempo de CPU, etc.

En la asignación variable, el sistema operativo intenta estimar la cantidad óptima de *frames* para cada proceso en cada momento.

- Si se asignan **menos *frames* de los necesarios**, se incrementan los fallos de página.
- Si se asignan **demasiados *frames***, se desaprovecha memoria.

Métodos de estimación

- **Modelo de conjunto de trabajo (Working Set Model)**
- **Algoritmo de frecuencia de fallos de página (Page Fault Frequency)**

Ámbito del reemplazo

Define si un proceso puede reemplazar sólo sus propias páginas o también las de otros procesos:

- **Reemplazo local:** sólo puede reemplazar páginas de otros procesos.
- **Reemplazo global:** puede reemplazar páginas de otros procesos.

Combinaciones comunes en sistemas modernos

Muchos sistemas operativos actuales como Linux o BSD Utilizan:

- **Asignación variable de frames**
- **Reemplazo global**

Esta combinación es fácil de implementar y permite aprovechar un **pool de *frames* libres** para cargar nuevas páginas sin demoras.

Caché de páginas

Los sistemas suelen mantener un **pool de *frames* libres** o **caché de páginas**.

Cuando se necesita reemplazar una página:

1. Se carga la nueva página en un *frame* de pool.
2. La página reemplazada se marca como libre y se añade nuevamente al pool.

3. Si esa página vuelve a ser referenciada y aún está en la caché, puede reutilizarse sin necesidad de ir al disco -> mejora el rendimiento.

Listas en la caché de páginas

- **Free Page List:** páginas robadas a los procesos, no modificadas.
- **Modified Page List:** páginas modificadas, deben escribirse al disco antes de liberarse.

Bloque de *frames* (*Frame locking*)

El sistema operativo puede proteger ciertos *frames* mediante un **bit de bloqueo**, lo que impide que las páginas que contienen sean reemplazadas.

Este mecanismo se utiliza para:

- Páginas del **kernel**.
- Estructuras críticas del sistema.
- Páginas involucradas en operaciones de E/S.

Generalmente no se aplica a páginas de usuario, ya que la E/S se realiza a través de **buffers del sistema** que sí están protegidos.

Algoritmos de reemplazo de páginas

Cuando se produce un fallo de página, ya sea en un esquema de reemplazo **local** o **global**, el sistema debe decidir **qué página residente se reemplazará**. Esta decisión está determinada por lo que se conoce como **algoritmo de reemplazo de páginas**.

Alcance del reemplazo

- **Reemplazo local:** sólo se consideran las páginas del proceso que provocó el fallo.
- **Reemplazo global:** se consideran las páginas de todos los procesos.

Los algoritmos descritos a continuación pueden aplicarse a ambos contextos.

Algoritmos clásicos de reemplazo

1. **Óptimo**
2. **LRU** (Least Recently Used)
3. **FIFO** (First-In First-Out)

De estos tres, sólo **FIFO** puede implementarse fácilmente.

El óptimo no es viable en la práctica, ya que requiere conocimiento futuro, y **LRU** necesita hardware especial para una implementación eficiente.

1. Optimal Algorithm

- Reemplaza la página que **tardará más en volver a ser referenciada**.
- Produce el **mínimo número posible de fallos de página**.
- No es implementable en la práctica, ya que implica conocer el futuro de las referencias.
- Se utiliza como **referencia teórica** para comparar otros algoritmos.

2. LRU

- Reemplaza la página que **no ha sido usada recientemente**.

- Equivalente al algoritmo óptimo, pero considerando el pasado en lugar del futuro.
- Se basa en el **principio de localidad temporal**: las páginas recientemente usadas tienden a ser reutilizadas pronto.

Implementaciones

- **Contadores**: cada página almacena el momento exacto en el que fue usada.
- **Listas**: las páginas se reorganizan en una lista según su uso reciente.

3. FIFO

- Reemplaza la **página más antigua en memoria**.
- Es fácil de implementar con una cola circular.
- **Problema**: sufre la **anomalía de Belady**, donde **más frames no siempre implican menos fallos**.

Stack Algorithms

Un **algoritmo de pila** garantiza que el conjunto de páginas residentes con N frames es **subconjunto** del que se tendría con $N+1$.

Los algoritmos óptimo y LRU son de pila, por lo que **no sufren la anomalía de Belady**.

Algoritmos de reemplazo aproximados

Dado que **LRU no puede implementarse directamente**, se utilizan algoritmos aproximados, aprovechando soporte de hardware que suele proporcionar:

- **Bit de referencia**
- **Bit de modificación**

Second chance algorithm

- Extiende FIFO utilizando el **bit de referencia**.
- Estructura: cola circular
- Cuando se debe reemplazar:
 1. Si el bit de referencia de la página apuntada es 0, se reemplaza.
 2. Si es 1, se limpia y se da una "**segunda oportunidad**", avanzando al siguiente *frame*.
- El **puntero que recorre la cola** funciona como una manecilla de reloj -> también llamado **Clock Algorithm**.

Variantes del Clock Algorithm

1. **Con bits** (referencia, modificación)
 1. Se busca una página con bits (0,0) para reemplazar.
 2. Si no se encuentra, se busca una con (0, 1), limpiando los bits de referencia la pasar.
 3. Si no se encuentra, se repite el proceso
2. **Doble índice**
 - Se utilizan **dos punteros**:
 - El primero limpia los bits de referencia.
 - El segundo identifica páginas que no fueron referenciadas desde el último pase.

- Se ejecuta **periódicamente**.
- Utilizando, por ejemplo el **algoritmo de page stealing en UNIX**.

Uso de bits de referencia adicionales

- El sistema operativo mantiene un **contador por página**.
- Periódicamente:
 - Lee el bit de referencia.
 - Inserta su valor a la izquierda del contador y desplaza a la derecha los anteriores.
- El **valor más bajo** representa la página menos recientemente usada.

Algoritmo	Implementación	Calidad	Anomalía de Belady
Optimal	No posible	Excelente	No
LRU	Difícil	Muy buena	No
FIFO	Muy fácil	Regular	Sí
Second Chance	Fácil	Buena	Depende
Clock Variantes	Moderada	Buena	No

Adaptación del tamaño del conjunto de residentes: alcance y asignación combinados.

La gestión del conjunto residente de un proceso implica decisiones sobre **el alcance de reemplazo** y **la asignación de *frames***. Al combinar ambas dimensiones, se derivan cuatro escenarios posibles:

Alcance de reemplazo:

- **Reemplazo global:** las páginas reemplazadas pueden pertenecer a cualquier proceso.
- **Reemplazo local:** sólo se reemplazan páginas del mismo proceso que provocó el fallo.

Asignación de *frames*:

- **Asignación fija:** el número de *frames* asignados del mismo proceso que provocó el fallo.
- **Reemplazo local:** el número de *frames* asignados a un proceso permanece constante.

Combinaciones posibles:

1. Reemplazo global con asignación fija
2. Reemplazo global con asignación variable
3. Reemplazo local con asignación variable
4. Reemplazo local con asignación fija

La opción 1 no es viable, ya que implica contradicción: si la asignación es fija, no tiene sentido permitir que el reemplazo sea global. Las otras combinaciones son posibles y presentan características particulares.

1. Reemplazo global con asignación variable

Es el enfoque adoptado por la mayoría de los sistemas operativos modernos como Linux, FreeBSD o Solaris.

- **Ventajas:** fácil de implementar y gestionar.
- Cuando un proceso genera un fallo de página, se le asigna un nuevo marco, y la página a reemplazar puede pertenecer a cualquier proceso.
- Se mantiene un **pool de frames libres**, y un proceso "page stealer"

2. Reemplazo local con asignación fija

- Cada proceso mantiene un conjunto residente constante durante su ejecución.
- **Desventajas:**
 - Si se asignan pocos *frames*: muchos fallos de página.
 - Si se asignan demasiados: desperdicio de memoria y baja multiprogramación.
- **No se usa en sistemas modernos** debido a su rigidez.

3. Reemplazo local con asignación variable

- El sistema ajusta dinámicamente el número de *frames* asignados al proceso según su comportamiento.
- Cuando ocurre un fallo de página:
 - Se reemplaza una página del mismo proceso.
 - El sistema evalúa si el conjunto residente actual es adecuado.
 - Si hay muchos fallos: se **incrementa** el número de *frames*.
 - Si hay pocos fallos: se **reduce** el número de *frames*.
- Para decidir estos ajustes, se pueden usar dos modelos:

Modelos para adaptar el conjunto residente

1. Modelo de conjunto de trabajo

- Basado en el principio de **localidad temporal**.
- El **Conjunto de trabajo** es el conjunto de páginas referenciadas entre los instantes t y $t + \Delta t$.
- Si es:
 - Muy pequeño: no captura toda la localidad.
 - Muy grande: mezcla varias localidades.
- El sistema intenta asignar a cada proceso el número de *frames* necesarios para contener su conjunto de trabajo actual.
- **Si no hay suficiente memoria** para alojar el conjunto de trabajo de un proceso, se **suspende su ejecución**.

Diferencia clave con LRU:

- = páginas **referenciadas** en las últimas referenciadas.
- LRU con *frames* = últimas **páginas cargadas**.

Implementación aproximada:

- Se puede usar un temporizador y un **bit de referencia** por página.
- Cada cierto número de referencias, los bits se reinician.

- Una página con bit activo se considera parte del conjunto de trabajo.

2. Frecuencia de fallos de página

- Método directo y eficiente para evitar el **thrashing**.
- Se establece un **límite de tiempo o referencias** entre fallos de página.
 - Si el tiempo entre fallos es **menor** que el límite, se añade un *frame*
 - Si es **mayor**, se **eliminan** páginas no referenciadas desde el último fallo.

Versión con doble umbral

Si el intervalo es:

- **Menor** al umbral inferior -> se **añade** un *frame*.
- **Mayor** al umbral superior -> se **eliminan** páginas.
- **Entre ambos** -> se hace un reemplazo local normal.

Comparación y conclusión

Modelo	Método	Reacción a PFF alto	Reacción a PFF bajo
Working Set	Basado en historial de acceso	Suspende si no hay memoria	Ajusta el conjunto de trabajo
Page-Fault Frequency	Basado en tiempo entre fallos	Asigna más marcos	Libera marcos

Segmentación bajo demanda

Cuando no se dispone de hardware de paginación, es posible implementar **memoria virtual mediante segmentación bajo demanda**. Para ello se requiere:

- Un **dispositivo de swap**.
- Hardware de segmentación que incluya un **bit de presencia**, el cual indica si un segmento está cargado en memoria.

El funcionamiento básico es el siguiente:

- Si un **segmento está en memoria** y se referencia, se accede de forma normal.
- Si un **segmento no está en memoria**, se lanza una **excepción** que provoca su carga desde el dispositivo de swap.

Algoritmo de reemplazo

Los algoritmos de reemplazo en segmentación bajo demanda son similares a los utilizados en paginación bajo demanda, aunque adaptados al tamaño variable de los segmentos.

En el caso de OS/2, el sistema mantenía una **lista de segmentos residentes en memoria**. De forma periódica:

- Los segmentos accedidos se movían al **final de la lista**.
- Se limpiaban los **bits de acceso**.

Cuando era necesario reemplazar un segmento, se elegía el **primer segmento** de la lista, lo que constituye una **aproximación a LRU**, ya que la lista se mantiene ordenada por **tiempo de último acceso**.

Segment replacement en segmentación bajo demanda

El mecanismo de reemplazo difiere del de paginación debido a que los segmentos **no tienen tamaño uniforme**. El procedimiento que seguía OS/2 ante un fallo de segmento era:

1. Se comprueba si hay suficiente espacio libre en memoria.
 - Si lo hay, se realiza una **compactación** para liberar espacio contiguo.
2. Si no hay espacio suficiente:
 - Se toma el **primer segmento** de la lista.
 - Si es necesario, se guarda en el **dispositivo de swap**.
3. Si ahora hay espacio, se carga el nuevo segmento en memoria, se actualiza la **tabla de segmentos** y se coloca el segmento cargado al final de la lista.
4. Si todavía no hay espacio suficiente, se **vuelve al paso 1**.

Otras consideraciones

Además del algoritmo de reemplazo, influyen en el rendimiento:

- El tipo de reemplazo: **local** o **global** uso.
- El tipo de asignación de *frames*: **fija** o **variable**.
- El **tamaño de página**:
 - **Páginas pequeñas**: mejor adaptación a la **localidad**.
 - **Páginas grandes**: simplifican el seguimiento y optimizan las **transferencias de disco**.

Colocación de datos en memoria

Aunque la paginación bajo demanda es **transparente** para el usuario y el programador, ciertos patrones pueden afectar significativamente al rendimiento.

En **lenguajes como C**, donde las matrices se almacenan **por filas**, recorrerlas **por filas** o **por columnas** puede tener efectos drásticos en la cantidad de **fallos de página**.

Procesos

¿Qué es un proceso?

Uno de los conceptos clave en un sistema operativo es el **proceso**.

Un **programa** es simplemente un conjunto de instrucciones.

Definición de proceso:

Un **proceso** es una **abstracción** que representa cada ejecución de un programa.

- **Importante**: un proceso no tiene que estar ejecutándose constantemente. Su **ciclo de vida** pasa por varias etapas, y la **ejecución** es sólo una de ellas.

Concepto de proceso

Podemos entender un proceso como:

- Cada instancia de un programa en ejecución.
- La entidad que el sistema operativo crea para ejecutar un programa.

Un proceso está formado por:

- **Espacio de direcciones:** conjunto de direcciones que el proceso puede usar.
- **Punto de control:** indica la siguiente instrucción que será ejecutada.

Algunos sistemas permiten que un proceso tenga **más de un punto de control**, lo que implica que el proceso puede ejecutar varias partes de su código al mismo tiempo. A esto se le llama **hilos (threads)**

Ejemplo práctico

- Si abrimos dos terminales y ejecutamos `ls -lR /` en una y `ls -lR /usr` en otra, tenemos **dos procesos distintos** ejecutando el **mismo programa** (`/bin/ls`).
- Cada uno ejecuta de forma **secuencial**; pero el sistema operativo va **alternando** entre ellos.
- Aunque parece que se ejecutan a la vez (**conurrencia**), en realidad sólo **uno se ejecuta en cada instante**. Esto se denomina **multitarea (multitasking)**.

En sistemas con **varias CPUs**, sí es posible tener **ejecución en paralelo real**, pero **cada CPU sólo ejecuta un proceso a la vez**.

Normalmente, el **número de procesos activos es mayor que el número de CPUs disponibles**. También existen procesos que son **concurrentes internamente**, mediante el uso de **hilos de ejecución**.

Estructura básica del espacio de direcciones de un proceso

El espacio de direcciones de un proceso se divide, en su forma más simple, en tres regiones:

- **Código:** contiene el código de todas las instrucciones del programa.
- **Datos:** incluye las variables globales y la memoria asignada dinámicamente (heap), que generalmente forma parte de esta región.
- **Pila:** se utiliza para:
 - El paso de parámetros
 - Guardar la dirección de retorno cuando se llama a una función.
 - Almacena variables locales de la función en ejecución.

Regiones adicionales

Además de estas tres, pueden existir otras regiones como:

- **Memoria compartida**
- **Librerías dinámicas enlazadas.**
- **Ficheros mapeados en memoria**, entre otros.

Para visualizar el espacio de direcciones de un proceso en Unix, se puede usar el comando `pmap`.

- Este comando muestra las **direcciones virtuales** (también llamadas **lógicas**) que usa el proceso.
- El espacio tiene "huecos", y **no refleja direcciones físicas reales** de memoria.

Procesos en UNIX

¿Qué es UNIX?

UNIX es un término genérico que se refiere a una familia de sistemas operativos. Existen muchas variantes o "sabores" de UNIX, tanto comerciales como libres, por ejemplo Linux, Solaris, AIX o FreeBSD. Cada variante puede o no cumplir con distintos estándares que regulan su funcionalidad, implementación o interfaz con el sistema operativo. Los principales estándares son: System V, BSD, POSIX.

¿Qué es el kernel?

El kernel es el núcleo del sistema operativo, el programa principal que controla el hardware y permite ejecutar procesos. Este núcleo reside en un archivo específico (como /unix, /vmlinuz, /bds, etc.) que se carga durante el arranque del sistema mediante un programa llamado bootloader.

El kernel se encarga de:

- Inicializar el sistema y crear el entorno necesario para ejecutar los procesos.
- Crear algunos procesos iniciales, entre ellos INIT (con PID 1), que es el primer proceso de usuario y el antecesor de todos los procesos de usuario en el sistema.
- Gestionar la interacción directa con el hardware.
- Proporcionar la interfaz de llamadas al sistema (system calls) que los procesos de usuario usan para interactuar con el kernel y, por tanto, con el hardware.

¿Cómo arranca una máquina UNIX?

Al encender o reiniciar la máquina comienza el procesador de arranque, que consta de cuatro fases principales:

1. **Firmware de las placas base (MB firmware):** El microprocesador ejecuta el código en una dirección fija, donde se encuentra el firmware (BIOS, UEFI, OpenBoot, etc). Este firmware realiza una serie de comprobaciones básicas (POST) y busca un dispositivo desde donde arrancar.
2. **Bootloader:** Es un programa pequeño y sencillo que el firmware carga. Su función es cargar el kernel del sistema operativo en memoria. La configuración del bootloaders define dónde está el kernel y qué parámetros pasarle.
3. **Kernel:** El kernel se carga en memoria, inicializa los dispositivos, configura las tablas internas (procesos, archivos abiertos...) y crea el primer proceso de usuario (INIT o systemd en algunas distribuciones Linux).
4. **Servicios:** INIT (o systemd) lee su configuración y ejecuta scripts para iniciar servicios del sistema (red, montaje de sistemas de archivos, daemons, servicios de login, etc.) dejando el sistema listo para su uso.

El proceso de arranque es similar en otros sistemas operativos, aunque las herramientas y procesos específicos varían.

¿Qué hace el kernel?

El kernel es el único proceso que corre directamente sobre el hardware del sistema. Los procesos de usuario no interactúan con el hardware directamente, sino a través de llamadas al sistema que el kernel gestiona.

Además, el kernel atiende las solicitudes de los dispositivos externos mediante interrupciones, que son señales enviadas al procesador para notificar eventos (por ejemplo, que una operación de E/S ha finalizado).

Modo kernel y modo usuario

Para funcionar correctamente, los sistemas operativos diferencian al menos dos modelos de ejecución:

- **Modo usuario:** Donde se ejecuta el código de los procesos de usuario.
- **Modo kernel:** Donde se ejecuta el código del sistema operativo.

Existen tres mecanismos principales que llevan a la ejecución en modo kernel:

1. **Llamadas al sistema:** cuando un proceso de usuario solicita un servicio del kernel.
2. **Excepciones:** situaciones anónimas como división por cero o errores de acceso a memoria.
3. **Interrupciones:** señales generadas por dispositivos hardware para avisar al kernel.

Algunas instrucciones del procesador sólo se pueden ejecutar en modo kernel.

Ejemplos de ejecución en modo kernel

Se puede observar el tiempo consumido en modo usuario y modo kernel con el comando `time`. Por ejemplo:

- Un programa que simplemente ejecuta un bucle infinito consume casi todo el tiempo en modo usuario.
- Si el programa hace llamadas al sistema (por ejemplo, `getpid()` en un bucle), el tiempo en modo kernel aumenta.
- Cuando un proceso se envía señales a sí mismo o a otros procesos, o provoca excepciones controladas, el tiempo en modo kernel crece notablemente.
- Interactuar con dispositivos (como mover el ratón o presionar teclas) también incrementa el tiempo en modo kernel.

Esto refleja que las operaciones que requieren interacción con el kernel o hardware se ejecutan en modo kernel, mientras que el resto del código del usuario se ejecuta en modo usuario.

Procesos e hilos (threads) en UNIX

En sistemas UNIX tradicionales, un proceso se define por:

- Espacio de direcciones: conjunto de direcciones de memoria que el proceso puede referenciar.
- Punto de control: Indica la siguiente instrucción que se va a ejecutar (contador de programa).

En sistemas UNIX modernos, un proceso puede tener varios puntos de control concurrentes, llamados **hilos**, que comparten el mismo espacio de direcciones pero pueden ejecutarse

simultáneamente.

Espacio de direcciones

Los procesos utilizan direcciones virtuales. Parte de este espacio corresponde al código y datos del kernel (llamado espacio del sistema o espacio kernel), al que sólo se puede acceder en modo kernel.

El kernel mantiene:

- Estructuras de datos globales.
- Estructuras específicas por proceso
- Registros de memoria que apuntan al espacio de direcciones del proceso actualmente en ejecución.

Kernel UNIX

El kernel está implementado en lenguaje C y está formado por:

- **Código kernel:** ejecutado en modo kernel, incluye el código para llamadas al sistema, manejadores de interrupciones y excepciones.
- **Datos kernel:** variables globales accesibles a todos los procesos (tabla de procesos, tabla de inodos, tabla de archivos abiertos...).
- **Pila kernel:** memoria usada como pila cuando se ejecuta código en modo kernel, para pasar parámetros y almacenar variables locales.

Kernel reentrante

El kernel UNIX es reentrante, lo que significa que:

- Varios procesos pueden estar ejecutando funciones del kernel al mismo tiempo, incluso la misma función simultáneamente.
- Para esto, el código del kernel debe ser de sólo lectura.
- Los datos globales del kernel deben estar protegidos contra accesos concurrentes.
- Cada proceso dispone de su propia pila de kernel.

Protección de datos kernel

En kernels tradicionales no preemptibles: Un proceso en modo kernel no puede ser interrumpido hasta que termine, bloquee o vuelva a modo usuario. Sólo ciertas estructuras de datos deben protegerse con flags simples.

En kernels preemptibles modernos: Un proceso en modo kernel puede ser interrumpido si otro proceso de mayor prioridad está listo para ejecutarse. Por ello, todas las estructuras de datos del kernel deben protegerse mediante mecanismos más complejos, como semáforos, especialmente en sistemas multiprocesador.

Estructuras de Datos del Sistema Operativo

¿Qué necesita el sistema operativo para gestionar los procesos?

Para poder gestionar adecuadamente los procesos, el sistema operativo necesita llevar a cabo varias tareas fundamentales. En primer lugar, debe asignar memoria para que el programa se pueda cargar y ejecutar. Esta es la base para que cualquier proceso comience a funcionar.

Además, como varios procesos pueden estar ejecutando el mismo programa, el sistema operativo debe ser capaz de distinguirlos entre sí. Para ello, se utilizan estructuras como el **descriptor de proceso** o el **identificador de proceso**, que permiten reconocer de forma única a cada uno.

Cuando un proceso no se está ejecutando activamente, es necesario conservar toda la información relacionada con su estado de ejecución, esto incluye los registros del procesador, la memoria que está utilizando y los recursos que tiene asignados.

Estructuras de Datos del Sistema Operativo

El sistema operativo debe tener conocimiento de todos los procesos que existen en el sistema y del estado en que se encuentra cada uno. Para esto, se utilizan **listas de descriptores de procesos**, organizadas generalmente por estado, o incluso por cada dispositivo de entrada/salida involucrado.

Bloque de Control del Sistema (SCB)

El Bloque de Control del Sistema, es un conjunto de estructuras de datos utilizadas por el sistema operativo para gestionar la ejecución de todos los procesos presentes en el sistema.

Contenido habitual del SCB:

- Una lista de todos los descriptores de procesos.
- Un puntero al proceso que está siendo ejecutado actualmente en la CPU.
- Punteros a listas de procesos según su estado: listos para ejecutarse, bloqueados en espera de dispositivos, etc.
- Un puntero a la lista de descriptores de recursos asignados.
- Referencias a las rutinas de gestión de interrupciones de hardware y software, así como a las rutinas de manejo de errores.

¿Cuándo entra en funcionamiento el sistema operativo?

El sistema operativo se ejecuta en ciertas circunstancias específicas, entre las que se incluyen:

- **Cuando ocurre una excepción:** por ejemplo, cuando un proceso intenta realizar una operación no permitida, como dividir por cero o acceder a una dirección de memoria no válida.
- **Cuando un proceso realiza una llamada al sistema:** por ejemplo cuando un disco ha terminado una operación de escritura pendiente.
- **Cuando un dispositivo externo requiere atención (interrupción):** por ejemplo, cuando un disco ha terminado una operación de escritura pendiente.
- **De forma periódica,** mediante una interrupción del reloj, que permite al SO retornar el control y tomar decisiones de planificación.

Bloque de Control de Proceso (PCB)

¿Qué es el PCB?

El sistema operativo mantiene una **tabla de procesos**, en la que guarda información sobre cada uno de ellos. Cada entrada de esa tabla representa un solo proceso, y está compuesta por su correspondiente.

Bloque de Control de Proceso

El PCB es una estructura esencial que contiene todos los datos necesarios para que el sistema operativo pueda gestionar adecuadamente un proceso en particular.

Información contenida en el PCB

Identificador del proceso:

- Identificador único del proceso.
- Identificador del proceso padre (en caso de que el sistema operativo implemente jerarquía de procesos).
- Identificadores de usuarios y de grupo asociados.

Información para la planificación (scheduling)

- Estado actual del proceso (ejecutando, listo, bloqueado).
- Evento que espera (en caso de estar bloqueado).
- Parámetros de planificación: prioridad del proceso y otros datos relevantes para el algoritmo de planificación.

Regiones de memoria asignadas:

- Región de datos.
- Región de código.
- Región de pila.

Recursos asignados:

- Archivos abiertos, representados mediante una tabla de descriptores de archivos o "file handlers".
- Puertos de comunicación asignados.

Organización en listas:

- Punteros que permiten organizar los procesos dentro de listas (lista de procesos listos o bloqueados).

Comunicación entre procesos:

- Información sobre colas de mensajes, memoria compartida, semáforos, señales, etc.

Estructuras de datos en UNIX

Implementación de un proceso

Para implementar el concepto de proceso, UNIX utiliza diversas estructuras que permiten la ejecución de un programa. En esencia, un proceso está compuesto por varios componentes clave:

- **Espacio de direcciones del usuario:** Contiene el código del programa, sus datos, la pila, regiones de memoria compartida y ficheros mapeados.
- **Información de control:** que permite al núcleo gestionar llamadas al sistema y otras funciones internas.
- **Mapas de traducción de direcciones:** necesarios para traducir direcciones virtuales físicas.
- **Credenciales:** identifican al usuario que ejecuta el proceso, determinando sus privilegios.
- **Variables de entorno:** otra vía para pasar información al proceso.
- **Contexto hardware:** registra el estado del hardware, como los registros del procesador. Este contexto se guarda en una zona especial de la `u_area` llamada **PCB (Process Control Block)** cuando se produce un cambio de contexto.

Aunque algunos de estos elementos son conceptualmente distintos, en la práctica comparten implementación. Por ejemplo, la pila del núcleo suele formar parte de la `u_area` y las credenciales del proceso se almacenan dentro de la estructura `proc`.

En sistemas Linux modernos, las estructuras `u_area` y `proc` han sido reemplazadas por una única estructura llamada `task_struct`.

La estructura `proc`

El núcleo del sistema mantiene una tabla de procesos, conocida como *process table*, compuesta por múltiples estructuras `proc`. Esta tabla se encuentra en el espacio del núcleo (kernel space), y cada entrada representa un proceso en el sistema.

Lo más relevante de esta estructura es que **permanece accesible en todo momento**, incluso cuando el proceso no está siendo ejecutado activamente por la CPU. Su contenido incluye toda la información que el núcleo necesita para gestionar un proceso.

Contenido relevante de `proc`

- Identificador del proceso (PID).
- Dirección de la `u_area`.
- Estado del proceso.
- Punteros a colas de listos, espera, etc.
- Prioridad del proceso y datos relacionados.
- Canal de sueño (sleep channel).
- Información sobre señales (máscaras, estados).
- Información sobre gestión de memoria.
- Punteros para mantener la estructura `proc` en colas de procesos disponibles, zombies, etc.
- Punteros a estructuras de hash basadas en el PID.
- Información sobre la jerarquía de procesos (procesos padre, hijos...).
- Flags o indicadores de estado.

`u_area`

La estructura `u_area` contiene información que sólo es relevante cuando el proceso se está ejecutando en la CPU. Aunque conceptualmente reside en el espacio de usuarios, **sólo es**

accesible desde el núcleo, y únicamente cuando el proceso está activo.

Siempre se sitúa en la misma dirección virtual, facilitando su acceso por parte del sistema operativo.

Contenido relevante a la `u_area`

- PBC
- Puntero a la estructura `proc`
- Parámetros y valores de retorno de llamadas al sistema.
- Información de señales y sus manejadores.
- Tabla de descriptores de ficheros del usuario.
- Punteros a los inodes del directorio raíz, directorio de trabajo y terminal asociada.
- La pila del núcleo del proceso.

Credenciales

Las credenciales permiten al sistema identificar qué acciones puede realizar un proceso, tanto sobre archivos como sobre otros procesos. Cada usuario del sistema se identifica mediante un número llamado **UID (user ID)** y cada grupo con un **GID (group ID)**.

El usuario especial **root** tiene UID 0, y dispone de privilegios completos: acceso a todos los ficheros, envío de señales a cualquier proceso, y capacidad de realizar llamadas al sistema privilegiadas.

Acceso a archivos

El acceso a un archivo está condicionado por:

- **El propietario del archivo**
- **El grupo asociado del archivo.**
- **Los permisos del archivo.**

Cuando un proceso intenta acceder a un archivo, el sistema compara sus credenciales con las del archivo:

1. Si el UID del proceso coincide con el del archivo, se aplican permisos de propietario.
2. Si no coincide el UID pero sí el GID, se aplican permisos de grupo.
3. Si no coincide ninguno, se aplican permisos para "otros".

Tipos de credenciales

Cada proceso posee **tres pares de credenciales**

- **Real:** Identifica al usuario original
- **Efectiva:** se usa para determinar los privilegios al acceder a archivos.
- **Guardada:** permite revertir cambios temporales en las credenciales.

Estas credenciales controlan también el envío y recepción de señales entre procesos, y se ven afectadas por las llamadas al sistema `setuid()` y `setgid()`.

Cambios en las credenciales

Existen tres mecanismos principales que pueden modificar las credenciales de un proceso:

1. `setuid(uid)` : cambia el UID efectivo de un proceso.
2. `setgid(gid)` :cambia el GID efectivo.
3. `exec()` : reemplaza el programa del proceso actual, pudiendo cambiar credenciales si el archivo ejecutado tiene ciertos permisos especiales.

También existen llamadas para consultar credenciales: `getuid()` , `geteuid` , `getgid()` , `getegid()` .

Sobre setuid y setgid

Estas llamadas pueden:

- Cambiar el UID o GID efectivo a su correspondiente valor real o guardado.
- Si el proceso tiene permisos de root (UID efectivo = 0), pueden cambiar las tres credenciales (real, efectiva y guardada).

Sobre exec()

Cuando un proceso ejecuta un programa con `exec()` , y el ejecutable tiene el bit *setuid* o *setgid* activado:

- **Setuid (s en los permisos del propietario)**: cambia el UID efectivo y guardado al UID del propietario del archivo.
- **Setgid (s en los permisos del grupo)**: cambia el GID efectivo y guardado al GID del archivo.

Variables de entorno

Las **variables de entorno** son cadenas de caracteres que permiten al sistema operativo y a los programas almacenar información relevante sobre el entorno de ejecución de un proceso. Son utilizadas ampliamente por shells, scripts, programas del sistema y aplicaciones para conocer datos como el directorio actual, la configuración de idioma, rutas a ejecutables, entre otros.

Forma y ubicación

- Las variables de entorno tiene la forma general de: **NOMBRE_VARIABLE=valor**
- Internamente, estas variables se almacenan al final de la **pila de usuario** en memoria.

Acceso a variables de entorno:

1. A través del tercer argumento de main:

La función main puede recibir un tercer argumento opcional: `int main (int argc, char *argv[], char *envp[])` Este tercer argumento, envp, es un array de punteros a cadenas que terminan en NULL. Cada cadena contiene una variable de entorno.

2. **Usando la variable global environ:**

También es posible acceder a las variables de entorno a través de una variable externa declarada como: `extern char **environ;` Al igual que envp, environ es un array de cadenas de caracteres finalizado con NULL.

3. **Mediante funciones de la biblioteca estándar:**

- `getenv("NOMBRE")`: Obtiene el valor de una variable.
- `putenv("NOMBRE=valor")`: Añade o modifica una variable.

- `setenv(nombre, valor, sobrescribir)` Añade o modifica de forma controlada.
- `unsetenv("NOMBRE")`: Elimina una variable de entorno.

Ciclo de vida de los procesos

Todo proceso en un sistema operativo comienza su existencia mediante una llamada al sistema para crear procesos. Esta función proporcionada por el sistema operativo, y el proceso que realiza esta llamada se convierte en el **proceso padre** del proceso recién creado.

Creación y ejecución

En algunos sistemas operativos, como Windows, la llamada para crear un proceso debe incluir también el archivo ejecutable que el nuevo proceso va a ejecutar. Sin embargo, en los sistemas tipo Unix, la creación de un proceso y la ejecución de un programa son operaciones separadas que se realizan mediante llamadas al sistema diferentes, como `fork()` y `exec()`.

Estados de un proceso

A lo largo de su vida, un proceso puede pasar por varios estados. Los principales son:

- **Ejecutándose (running)**: el proceso está siendo ejecutado actualmente por la CPU.
- **Listo para ejecutar (ready to run)**: el proceso está preparado para ejecutarse pero está esperando que la CPU lo seleccione.
- **Bloqueado**: el proceso está esperando un evento externo, como la finalización de una operación de entrada/salida.

Cuando finaliza, un proceso realiza una llamada al sistema para terminar su ejecución. Este es el punto final de su ciclo de vida, y se encarga de liberar los recursos asignados al proceso.

Sistemas con intercambio (swapping)

Sistemas antiguos: intercambio completo

En los sistemas más antiguos, parte de la memoria secundaria (como el disco) se utilizaba para **intercambiar procesos completos** fuera de la memoria principal con el objetivo de aumentar el grado de multiprogramación (número de procesos activos al mismo tiempo). Esta técnica se conoce como **swapping**, y cuando un proceso era intercambiado fuera de la memoria, pasaba al estado de **intercambiado** o **suspendido**. Las transiciones asociadas a este estado son:

- **Swap in**: el proceso se vuelve a cargar en la memoria principal.
- **Swap out**: el proceso es desplazado fuera de la memoria.

Los sistemas que implementaban esta técnica se conocen como **sistemas con intercambio**.

Sistemas modernos: intercambio por páginas

En los sistemas actuales, el intercambio se realiza **por piezas** de los procesos, en lugar de intercambiar el proceso completo. Estas piezas suelen ser páginas de memoria. De esta forma, es posible ejecutar procesos que no están completamente cargados en memoria o incluso procesos más grandes que la propia memoria física disponible.

El espacio en disco utilizado puede ser una **participación dedicada** o un archivo de

intercambio dentro del sistema de archivos. Este modelo se basa en el concepto de **memoria virtual**, y los sistemas que lo utilizan se conocen como **sistemas con paginación**.

Estados de los procesos

La terminología común utilizada para describir los estados de un proceso incluye:

- **En ejecución:** el proceso está usando la CPU.
- **Listo:** el proceso puede ejecutarse tan pronto como se le asigne CPU.
- **Bloqueado:** el proceso espera que ocurra un evento externo.
- **Intercambiado:** el proceso ha sido desplazado a la memoria secundaria, ya sea desde el estado listo o bloqueado.

Transiciones de estado

Entrada al estado de ejecución

El proceso que está primero en la cola de listos es seleccionado por el planificador para ejecutarse.

Entrada al estado listo para ejecutar

1. Se ha creado un proceso nuevo.
2. Un proceso que estaba ejecutándose es interrumpido por el sistema operativo mediante una conmutación de contexto (preempted).
3. Un proceso bloqueado se desbloquea cuando ocurre el evento esperado (con la finalización de una entrada/salida). Esto se llama **desbloqueo** o **despertar**.
4. Un proceso intercambiado (en estado listo) es traído de nuevo a memoria principal.

Entrada al estado bloqueado:

- Desde ejecución: el proceso hace una llamada al sistema que no puede completarse de inmediato y se bloquea.
- Desde bloqueado/intercambiado: el sistema operativo puede decidir traer a memoria un proceso bloqueado que estaba intercambiado, aunque no todos los sistemas permiten esta transición.

Un proceso también puede pasar a los estados bloqueado/intercambiado o listo/intercambiado si el sistema operativo decide liberarlo de la memoria principal para hacer espacio a otros procesos.

Creación de procesos

La **creación de procesos** es un servicio del sistema operativo. Cuando un proceso invoca la llamada al sistema correspondiente, el sistema operativo realiza las siguientes acciones:

1. Asigna un identificador único al nuevo proceso.
2. Crea e inicializa su PCB, la estructura de datos que contiene toda la información del proceso.
3. Actualiza el SCB, añadiendo el nuevo proceso a su tabla.
4. Asigna memoria y, si es necesario, carga en ella el programa que se va a ejecutar.
5. Coloca el nuevo proceso en la cola de listos.

Modelos de creación

Un proceso puede crearse en diversos contextos, aunque internamente siempre se utiliza la misma llamada al sistema:

- **Inicialización del sistema:** durante el arranque del sistema se crean múltiples procesos.
 - Algunos interactúan directamente con el usuario.
 - Otros no tienen interacción directa y se encargan de servicios del sistema. Estos se conocen como **demonios (daemons)**
- **Llamadas del propio código del proceso:** por ejemplo, un programa que crea nuevos procesos para rellenar un búfer con datos.
- **Acción explícita del usuario:** al ejecutar un comando en una terminal o hacer click sobre un icono gráfico, es el código del entorno el que crea el nuevo proceso.
- **Procesamiento por lotes (batch).**

Terminación de procesos

Cuando un proceso finaliza, lo hace mediante una llamada al sistema que invoca el servicio de **terminación de proceso**.

Acciones del sistema operativo al finalizar un proceso:

- Elimina el PCB del proceso.
- Libera todos los recursos que estaban asignados al proceso.
- Si el proceso tiene **procesos hijos**, puede optar por:
 - Esperar a que estos terminen.
 - Terminarlos explícitamente.
 - Dejarlos ejecutarse independientemente.

Tipos de terminación

Existen dos formas principales de finalización:

1. **Terminación normal:** el proceso realiza de una llamada a la función de terminación `exit()`.
2. **Terminación anormal:** no está prevista por el código del procesador. Puede deberse a errores, señales externas o fallos del sistema. En cualquier caso, el proceso es forzado a terminar.

Ciclo de vida de los procesos en UNIX

En UNIX, un **proceso** es una instancia de un programa en ejecución. El sistema operativo crea un proceso con el propósito de ejecutar un programa, proporcionando un entorno de ejecución que incluye un espacio de direcciones y uno o varios puntos de control para gestionarlo.

Cada proceso tiene una vida útil definida: **se crea mediante la llamada al sistema `fork()` o `vfork()` y termina cuando se invoca `exit()`**. Durante su ejecución, un proceso puede iniciar otro programa mediante alguna de las variantes de `exec()`.

Además, todo proceso en UNIX **tiene un proceso padre**, y puede tener uno o más **procesos hijos**. Esta estructura forma un árbol de procesos, cuyo ancestro común es usualmente el

proceso `init` . Si un proceso padre termina antes que sus hijos, estos son adoptados por `init` .

Estados de los procesos (System V R2)

Un proceso puede encontrarse en distintos **estados**, que reflejan su situación en el sistema:

- **Idle**. El proceso está en creación, pero aún no está listo para ejecutarse.
- **Runnable**. Está listo y puede ser seleccionado por el planificador.
- **Asleep**. Está esperando que ocurra algún evento externo. Puede estar en memoria principal o en el área de intercambio (swap).
- **User running**. El proceso se está ejecutando en modo usuario.
- **Kernel running**. El proceso se está ejecutando en modo núcleo.
- **Zombie**. El proceso ha finalizado pero su proceso padre aún no ha recogido su estado con una llamada a `wait()` , por lo que su estructura interna (`proc`) no ha sido liberada

Desde la versión 4.2BSD se introdujo un nuevo estado: **stopped**, que puede ser:

- **stopped-runnable**
- **stopped-blocked**

Este estado se alcanza cuando el proceso recibe señales como:

- SIGSTOP: detiene el proceso sin posibilidad de ignorarla
- SIGTSTP: se activa con ctrl+z, deteniendo el proceso en primer plano.
- SIGTTIN y SIGTTOU: cuando un proceso en segundo plano intenta leer o escribir en el terminal.

Para reanudar un proceso se usa SIGCONT.

Además, existen comandos de terminal para gestionar estos procesos:

- `jobs` : lista los trabajos del terminal.
- `fg` : mueve un trabajo al primer plano.
- `bg` lo envía al segundo plano.

Cambios de estado del proceso

- La ejecución **siempre comienza y termina en modo kernel**.
- La transición a estado **bloqueado** ocurre desde el modo kernel.
- Las transiciones hacia o desde el estado **runnable** también se gestionan en modo kernel.
- Al terminar un proceso, entra en estado zombie hasta que el proceso padre realice una llamada a `wait()` .

Llamadas al sistema

`fork()`

La llamada a `fork()` crea un nuevo proceso: **un clon del proceso padre**, con una copia exacta de su espacio de direcciones. La única diferencia entre padre e hijo es el valor que devuelve `fork()`:

- En el proceso hijo devuelve 0
- En el proceso padre devuelve el PID del hijo.

`exec()`

Una vez creado un proceso, este puede reemplazar su contenido con otro programa usando alguna variante de `exec()`. Este reemplazo afecta completamente al código, datos, pila y demás elementos del espacio de direcciones del proceso.

Es importante remarcar que `exec()` **no crea un nuevo proceso**. El proceso que invoca `exec()` sigue siendo el mismo con el mismo PID, pero con un contenido completamente diferente.

`exit()`

Cuando un proceso termina su ejecución, llama a `exit()`, que libera recursos y marca al proceso como zombie hasta que su padre recoja su estado.

Ejemplo de uso combinado de `fork()` y `exec()`

```
if ((pid=fork())==0)
{ // Proceso hijo
    if (execv("./programilla", args) == -1)
    {
        perror("fallo en exec");
        exit(1);
    }
}
else if (pid < 0)
{
    perror("fallo en fork");
}
else
{
    // Proceso padre continúa
}
```

Tareas relacionadas por `fork()`

1. Asigna espacio de intercambio (swap).
2. Asigna un PID y estructura `proc`.
3. Inicializa la estructura `proc`.
4. Asigna mapas de traducción de direcciones.
5. Asigna y copia `u_area` desde el padre.
6. Actualiza campos en `u_area`.
7. Comparte la región de código del padre.
8. Duplica segmentos de datos y pila.
9. Inicializa el contexto hardware.
10. Cambia el estado del hijo a "listo para ejecutarse".
11. Devuelve 0 al hijo.
12. Devuelve el PID del hijo al padre.

Optimización de `fork()`

En muchos casos, un proceso hijo creado por `fork()` ejecuta inmediatamente un nuevo programa con `exec()`. Esto provoca que el trabajo hecho por `fork()` (como copiar datos y pila) se desaproveche, ya que `exec()` reemplaza todo el contenido del proceso.

Para optimizar esta situación, se introducen dos técnicas:

1. Copy-on-write (COW)

- El hijo y el padre **comparten** los datos y la pila.
- Estos se marcan como **sólo lectura**.
- Cuando uno de los procesos intenta modificarlos, se genera una excepción y **sólo se copia la página modificada**.

2. `vfork()`

- Utilizado cuando se planea usar `exec()` inmediatamente después de crear el proceso.
- El hijo **no copia nada**: ni datos ni pila.
- El hijo ejecuta en el espacio de direcciones del padre, que queda **bloqueado** hasta que el hijo llame a `exec()` o `exit()`.

Ejecutar programas con `exec()`

Al invocar `exec()`:

1. El espacio de direcciones actual se descarta.
2. Se carga el nuevo programa.
3. Se copian los argumentos y el entorno a la nueva pila de usuario.
4. Se reinician los manejadores de señales.
5. El contador de programa se sitúa en la primera instrucción del nuevo programa.

`exec()` **no cambia el PID ni la estructura del proceso**, pero si el ejecutable tiene activados los bits `setuid` o `setgid`, se modifican las credenciales efectivas del proceso.

Tareas realizadas por `exec`

1. Obtiene el archivo ejecutable.
2. Verifica los permisos de ejecución.
3. Comprueba que es un archivo ejecutable válido.
4. Si empieza por `#!`, invoca el intérprete correspondiente.
5. Ajusta credenciales si hay bits `setuid/setgid`.
6. Guarda argumentos y entorno en espacio del kernel.
7. Libera el espacio de direcciones anterior.
8. Asigna uno nuevo y carga el programa.
9. Copia argumentos y entorno a la nueva pila.
10. Restaura manejadores de señales por defecto.
11. Inicializa el contexto hardware y apunta al inicio del programa.

Finalización de un proceso: `exit()`

Al terminar, un proceso llama a `exit()` y realiza las siguientes tareas:

- Desactiva señales.
- Cierra archivos abiertos.
- Libera entradas en tablas de nodos virtuales.
- Guarda estadísticas de uso y estado de salida de su `proc`.
- Cambia su estado a zombie (SZOMB).
- Sus hijos pasan a ser hijos de `init`.
- Libera espacio de direcciones, `u_area` y espacio de intercambio.
- Envía SIGCHLD al proceso padre
- Si el padre está esperando, se despierta.
- Realiza un cambio de contexto (switch).

El proceso sigue existiendo como zombie hasta que su padre use `wait()`.

Esperar la terminación de un hijo: `wait()`

Cuando un proceso desea saber cómo ha terminado su hijo, puede invocar `wait()` o una de sus variantes:

- `wait()` se bloquea hasta que termina cualquier hijo.
- Retorna el PID del hijo terminando y su código de salida.
- Libera la estructura `proc` del hijo.

Variantes como `waitpid()`, `waitid()`, `wait3()` o `wait4()` permiten opciones como:

- `WNOHANG` : no espera si el hijo aún no ha terminado.
- `WUNTRACED` : informa también si el hijo ha sido detenido.
- `WCONTINUED` : informa si un hijo detenido ha continuado.
- `WNOWAIT` : no libera la estructura `proc`

Sin una llamada a `wait()`, el proceso queda como zombie. para evitarlo se puede usar el flag `SA_NOCLDWAIT` en `sigaction`.

Interpretación del valor devuelto

El valor pasado como parámetro a `wait (int *stat_loc)` contiene:

- Si el proceso terminó con `exit()` : los 8 bits menos significativos son 0, y los siguientes 8 contienen el valor devuelto.
- Si terminó por una señal: 8 bits menos significativos son 0, y los inferiores contienen el número de la señal.
- Si fue detenido: contiene una marca especial (`WSTOPFLG`) y el número de señal que lo detuvo.

Planificación de procesos

En un sistema operativo multiprogramado, varios procesos, y en muchos casos, hilos, compiten constantemente por el uso de la CPU. Cada proceso está compuesto por una sucesión de ráfagas de CPU y ráfagas de entrada/salida (E/S), comenzando y finalizando siempre con una ráfaga de CPU.

El componente encargado de decidir cuál de los procesos preparados para ejecutarse obtiene el control de la CPU es el **planificador**

Algoritmos de planificación

Existen dos tipos principales de algoritmos de planificación, depende cómo gestionan la asignación de la CPU:

- **Algoritmos no expropiativos:** el proceso que está ejecutándose en la CPU permanece en ella hasta que finaliza su ráfaga de CPU de forma voluntaria. Esto puede deberse, por ejemplo, al inicio de una operación de E/S, a la espera del fin de un proceso hijo o simplemente porque el proceso ha terminado.
- **Algoritmos expropiativos:** el planificador puede interrumpir al proceso en ejecución y retirarlo de la CPU, incluso aunque no haya terminado su ráfaga. Esto permite una mayor capacidad de respuesta, especialmente en entornos interactivos.

Tipos de planificadores

En un sistema operativo pueden intervenir distintos tipos de planificadores, según el momento y el nivel del sistema en que actúan:

- **Planificador a corto plazo:** selecciona, entre los procesos listos para ejecutarse, cuál accede a la CPU. Es el que se invoca con más frecuencia.
- **Planificador a medio plazo:** presente en sistemas con intercambio, swapping, decide qué procesos que han sido sacados de memoria serán reintroducidos en ella.
- **Planificador a largo plazo:** en sistemas por lotes, se encarga de decidir qué procesos en la cola de espera del dispositivo de spool se cargarán en memoria principal. Controla el grado de multiprogramación del sistema.

Objetivos de la planificación

Los objetos de un planificador pueden variar dependiendo del entorno en el que se aplique. Algunos de los más relevantes son:

Sistemas por lotes (batch)

- Suele utilizarse planificación no expropiativa.
- El planificador a largo plazo es fundamental.
- Se prioriza la **eficiencia** y un alto **rendimiento**, es decir, ejecutar el mayor número de trabajos posible por unidad de tiempo.
- Otros objetivos incluyen minizar el **tiempo de retorno** y maximizar el **uso de la CPU**

Sistemas interactivos

- Predominan los algoritmos expropiativos, especialmente con planificación por **compartición de tiempo**.
- El objetivo principal es que todos los procesos reciban CPU en un intervalo razonable, favoreciendo una **buena capacidad de respuesta**.
- También se busca la **proporcionalidad**, es decir, que tareas sencillas obtengan tiempos de respuesta cortos.
- Se intenta maximizar el número de usuarios activos interactuando con el sistema.

Sistemas en tiempo real

- Algunos procesos tienen **restricciones temporales estrictas** que deben cumplirse.
- Se suele aplicar planificación basada en **prioridades**, asignando las más altas a los procesos con requerimientos más exigentes.
- Los objetivos fundamentales son la **fiabilidad** y la **previsibilidad**.

Criterios típicos de evaluación

Aunque cada sistema tiene sus propios objetivos específicos, existen ciertos criterios comunes:

- **Equidad**: todos los procesos deben tener acceso razonable a la CPU.
- **Política**: se deben respetar criterios previamente definidos.
- **Balance**: lograr que las distintas partes del sistema trabajen con cargas similares.

En sistemas por lotes

- **Rendimiento**: número de trabajos completados por unidad de tiempo. Cuanto mayor, mejor.
- **Tiempo de retorno**: tiempo total desde que se envía un trabajo hasta que termina. Se busca minimizarlo.
- **Tasa de uso de CPU**: mantener la CPU ocupada el mayor tiempo posible.

En sistemas de tiempo compartido

- **Tiempo de respuesta**: tiempo desde que el usuario lanza una tarea hasta que recibe una primera respuesta.
- **Proporcionalidad**: las tareas simples deben responder rápidamente.
- **Maximización de usuarios activos**: permitir que muchos usuarios trabajen de forma interactiva.

En sistemas en tiempo real

- **Fiabilidad**: evitar pérdidas de datos y cumplir límites temporales.
- **Previsibilidad**: asegurar que el comportamiento temporal del sistema sea constante y controlable.

Evaluación de un algoritmo de planificación

La evaluación de un algoritmo de planificación busca determinar cuál se adapta mejor a las necesidades de un sistema. Dado que los objetivos pueden variar, existen tres enfoques principales:

Modelos deterministas

Se parte de una carga de trabajo concreta y se analiza cómo se comporta el sistema. Por ejemplo, se puede calcular el tiempo medio de retorno, el rendimiento, etc.

Ventajas:

- Simples de aplicar
- Ofrecen resultados exactos.

Desventajas:

- Pueden ser engañosos si la carga de trabajo no representa adecuadamente la realidad del sistema

Modelos no deterministas

En muchos sistemas no se puede predecir ni el momento de llegada ni la duración de los procesos. Por tanto, se utilizan funciones de distribución de probabilidad para modelar estos factores.

Con estas distribuciones se pueden estimar valores medidos como:

- Tiempo de espera.
- Tiempo de retorno.
- Uso de CPU.

El sistema se modela como un conjunto de **servidores con colas de espera**, donde la CPU es uno de estos servidores y cada dispositivo de E/S puede ser otro.

Si se conoce la tasa de llegada de procesos y su duración, se pueden calcular métricas como:

- Carga del servidor.
- Longitud media de las colas.

Simulación

Otra opción consiste en simular el comportamiento del sistema con un conjunto de procesos.

Ventajas:

- Proporcionan una visión bastante realista del comportamiento del algoritmo.

Desventajas:

- Elevado coste computacional.

Implementación

Consiste en implementar directamente el algoritmo en un sistema real para observar su rendimiento.

Ventajas

- Los datos obtenidos son reales

Desventajas

La simple presencia de un nuevo algoritmo puede alterar el comportamiento de los usuarios y, por tanto, distorsionar los resultados.

Medición temporal del procesos

Para evaluar el comportamiento de los algoritmos, se utilizan varias métricas relacionadas con el tiempo:

- **Tiempo de retorno:** es el tiempo desde que el proceso comienza hasta que termina.
- **Tiempo de espera:** se obtiene restando al tiempo de retorno el tiempo de CPU y el de E/S.
- **Índice de servicio:** proporción entre el tiempo de servicio y el tiempo total de retorno:

Planificación no expropiativa

FCFS: First-Come, First-Served

Este algoritmo es uno de los más simples en la planificación de procesos. Su funcionamiento se basa en una estructura de tipo FIFO, es decir, los procesos se ejecutan en el orden en que llegan.

Ventajas:

- Es fácil de implementar, ya que basta con una cola sencilla.
- Resulta justo en el sentido de que los procesos se ejecutan por orden de llegada.

Inconveniente principal:

Tiene un bajo rendimiento en términos de **throughput** debido al fenómeno llamado "efecto convoy", que ocurre cuando un proceso requiere mucho tiempo de CPU bloquea a muchos procesos más cortos que llegan después. Por ejemplo, un proceso intensivo en CPU puede retrasar a varios procesos que sólo necesitan hacer operaciones de entrada/salida.

SJF: Shortest Job First

El algoritmo de planificación SJF selecciona el proceso con la ráfaga de CPU más corta entre los que están listos para ejecutarse. Este enfoque reduce al mínimo el tiempo medio de retorno **cuando todos los procesos llegan al mismo tiempo-**

Ventajas:

- Produce tiempos de retorno muy bajos, siendo óptimo en el caso ideal en que se conocen todas las ráfagas de CPU de antemano.

Limitaciones

- Es un algoritmo teórico, ya que en la práctica no se conoce con certeza cuánto durará la ráfaga de CPU de un proceso.
- Al ser no expropiativo, los resultados dependen mucho del orden de llegada de los procesos.

SPN: Shortest Process Next

Suele implementarse mediante una **estimación de la próxima ráfaga de CPU**, usando la fórmula:

Donde:

- es la próxima estimación.
- es la duración real de la última ráfaga.
- es la estimación anterior.
- es un factor de ajuste entre 0 y 1.

Planificación por prioridades no expropiativa

SJF puede considerarse un caso particular de planificación por prioridades, en el que las prioridades se asignan según la duración estimada de la ráfaga de CPU.

Tipos de prioridades:

- **Internas:** calculadas por el sistema operativo, basadas en características como uso de CPU, memoria, archivos abiertos, etc.
- **Externas:** asignadas por el usuario o el administrador del sistema.
- **Mixtas:** combinación de ambas.

Además, las prioridades pueden ser:

- **Estáticas:** no cambian a lo largo del tiempo.
- **Dinámicas:** el sistema las modifica automáticamente según el comportamiento de los procesos.

Problema importante: la **iniciación**, donde los procesos con baja prioridad pueden no llegar a ejecutarse. Esto suele mitigarse mediante:

- **Aging:** aumento progresivo de la prioridad de un proceso cuanto más tiempo pase esperando.
- Prioridades basadas en el cociente entre tiempo de espera y uso de CPU.

Planificación Expropiativa

Prioridades Expropiativas

A diferencia de la versión no expropiativa, aquí **un proceso puede ser interrumpido** si otro con prioridad superior se vuelve elegible.

- El proceso que estaba ejecutándose entra en estado de *ready*, y el nuevo proceso toma la CPU.
- Las mismas clasificaciones de prioridad (interna, externa, dinámica...) se aplican aquí.
- El rendimiento de este método depende de cómo se asignen las prioridades.

SRTF: Shortest Remaining Time First

Este es el equivalente expropiativo de SJF. Cuando un nuevo proceso llega, su tiempo estimado de ráfaga se compara con el tiempo restante del proceso que está en ejecución. Si el nuevo proceso tiene una ráfaga más corta, interrumpe al actual. Este algoritmo también requiere conocer, o estimar, las ráfagas de CPU.

Round-Robin

Round-Robin asigna a cada proceso un **quantum** de tiempo máximo en CPU. Si un proceso agota su quantum, se interrumpe su ejecución, se coloca al final de la cola de procesos listos, y el siguiente proceso en la cola toma el control.

Ventajas

- Fácil de implementar.
- Muy justo: todos los procesos obtienen su "porción" de tiempo.

Desventajas

- Si el quantum es muy pequeño, se generan muchos cambios de contexto, lo que desperdicia recursos.
- Si es demasiado grande, el algoritmo se comporta como FCFS.

Regla práctica: se obtiene buenos resultados si el **80% de las ráfagas de CPU son más cortas que el quantum**. Valores típicos oscilan entre **20 y 50 ms**.

Multilevel Queue

Este algoritmo extiende la planificación por prioridades dividiendo los procesos en diferentes colas según su prioridad. Cada cola puede tener su propio algoritmo de planificación.

- Para evitar inanición, se emplean **prioridades dinámicas**, permitiendo que un proceso cambie de cola.

Planificación en Sistemas de Tiempo Real

En sistemas de tiempo real, el tiempo de respuesta es fundamental. Los estímulos generados por dispositivos físicos requieren respuestas dentro de un tiempo limitado.

Tipos:

- **Hard real-time:** todos los límites de tiempo deben cumplirse.
- **Soft real-time:** pueden tolerarse algunos retrasos ocasionales.

Los sistemas de tiempo real suelen gestionar **eventos periódicos** y **eventos no periódicos**.

Condición de planificabilidad:

Un sistema con m flujos periódicos es planificable si:

- : periodo del flujo .
- : periodo del flujo .

Planificación de Hilos

Un **hilo** es la unidad básica de ejecución de un proceso. Mientras que un proceso puede tener uno o más hilos, todos los hilos de un mismo proceso **comparten el espacio de direcciones** y los recursos del proceso.

Cada hilo posee su:

- Identificador.
- Contador de programa.
- Registros.
- Pila.

Ventajas

- Mayor capacidad de respuesta: si un hilo se bloquea, otros pueden seguir ejecutándose.
- Menor coste de creación y cambio de contexto que los procesos.
- Aprovecha arquitecturas multiprocesador.

La planificación se produce a dos niveles:

1. Se selecciona un proceso.
2. Dentro de ese proceso, se selecciona un hilo.

Planificación en Multiprocesadores

En sistemas con varios procesadores, la planificación se complica.

Tipos de planificación:

- **Asimétrica:** un procesador se encarga de toda la planificación, los demás sólo ejecutan.
- **Simétrica:** cada procesador realiza su propia planificación.

En algunos sistemas, los procesadores comparten la cola de procesos listos. Esto implica tener cuidado para que un mismo proceso no sea asignado a más de un procesador.

Afinidad de caché

Cuando un hilo se ejecuta durante mucho tiempo en un procesador, su caché contiene datos relevantes. Volver a ejecutar ese hilo en el mismo procesador mejora el rendimiento.

Algoritmos de afinidad:

1. Asignan grupos de hilos a cada procesador.
2. Cada procesador planifica dentro de su grupo.

Ventajas

- Aprovechamiento de la caché.
- Menor coste de migración.

Inconvenientes

- Riesgo de dejar procesadores sin carga.

Balanceo de carga

Para evitar desequilibrios:

- **Migración forzada:** el sistema fuerza la redistribución de procesos cuando detecta que algunos procesadores están sobrecargados.
- **Migración voluntaria:** los hilos o procesadores pueden moverse de forma controlada si hay beneficio.

Planificación de procesos en UNIX

Planificación tradicional en UNIX

El sistema tradicional de planificación en Unix emplea un modelo de prioridades expropiativo, en el que siempre se ejecuta el proceso con mayor prioridad. En este contexto, una prioridad más alta se representa con un número menor. Esta prioridad se ajusta dinámicamente dependiendo del comportamiento del proceso:

- Si un proceso utiliza mucho tiempo de CPU, su prioridad disminuye.
- Si un proceso permanece mucho tiempo en la cola de listos, su prioridad aumenta.

Cuando aparece un proceso con mayor prioridad que el que actualmente ocupa la CPU, el sistema lo interrumpe, salvo que el proceso actual esté ejecutándose en modo kernel. En ese caso, la interrupción se pospone hasta que vuelva al modo usuario, termine o se bloquee. Los procesos con la misma prioridad comparten la CPU en régimen de "round-robin".

Factores de Recalculación de Prioridad

Las prioridades en modo usuario se recalculan teniendo en cuenta dos factores:

- **El factor "nice"**: modificable mediante la syscall `nice()`, permite al usuario influir en la prioridad.
- **Uso reciente de CPU**: cuanto más CPU haya usado un proceso recientemente, menor será su prioridad.

La estructura `proc` del núcleo contiene varios campos relevantes para esta planificación:

- `p_cpu`: refleja el uso de CPU reciente.
- `p_nice`: almacena el valor de "nice".
- `p_usrpri`: prioridad en modo usuario, calculada a partir de `p_cpu` y `p_nice`
- `p_pri`: prioridad efectiva usada en la planificación.

En modo usuario, `p_pri` es igual a `p_usrpri`. Sin embargo, cuando un proceso despierta tras haber estado bloqueado, su prioridad (`p_pri`) se ajusta el motivo del bloqueo, lo que se conoce como **prioridad en modo kernel** o **prioridad de sueño**. Estas prioridades de kernel son más altas que las prioridades de usuario, con el fin de completar llamadas al sistema lo más rápido posible.

Recalculación de Prioridades en Modo Usuario

Cada "tick" de reloj, el sistema incrementa `p_cpu` para el proceso en ejecución.

Cada segundo, una rutina (`schedcpu()`) recalcula `p_cpu` y la prioridad de usuario:

- **En BSD:**
 -
 -
- **En System V R3:**
 -
 -

En ambos casos, PUSER es un valor base que asegura que las prioridades de usuario sean más bajas que las de kernel.

Implementación de la Planificación Tradicional

La planificación se implementa mediante un **array de colas multinivel**, típicamente 32 colas, cada una correspondiente a un rango de prioridades adyacentes. Cuando se recalcula la prioridad de un proceso, este se mueve a la cola apropiada.

- La función `switch()` carga el primer proceso de la primera cola no vacía.
- Cada 100 ms, `roundrobi()` fuerza el cambio al siguiente proceso en la misma cola.

Cambios de Contexto

Se produce un **cambio de contexto** en tres situaciones:

1. **Voluntario**: el proceso termina o se bloquea (`sleep()` o `exit()` llaman a `switch()`).
2. **Involuntario**:
 - Tras recalcular prioridades aparece un proceso con mayor prioridad.
 - Un proceso o rutina de interrupción despierta a un procesador de mayor prioridad.

En estos casos, si el sistema está en modo kernel, se utiliza un flag `runrun` para indicar que debe realizarse un cambio al volver al modo usuario.

Limitaciones de la planificación tradicional

Este modelo presenta varias **deficiencias**:

- No se adapta bien a sistemas con muchos procesos o procesadores.
- No garantiza una cuota mínima de CPU para un proceso o grupo.
- No asegura un tiempo de respuesta mínimo.
- Existe la posibilidad de **inversión de prioridad**.

Inversión de prioridad

Este fenómeno ocurre cuando un proceso de baja prioridad impide la ejecución de uno de mayor prioridad.

Planificación en System V Release 4

SVR4 introduce un modelo más sofisticado y extensible:

- Se separa la política de planificación del mecanismo de implementación.
- Permite implementar nuevas políticas.
- Incluye procesos en tiempo real.
- Evita la inversión de prioridad mediante herencia de prioridad.

Clases de planificación

Los procesos se agrupan en **clases de planificación**, cada una con su propia política. Algunas clases comunes:

- TS, Time Sharing, compartición de tiempo
- SYS, System, procesos del sistema
- RT, Real Time, procesos de tiempo real
- IA, Interactive, versión mejorada de TS

Consulta de clases en el sistema: `dispadmin -l`

Rango de prioridades

- 0-59: **TS**
- 60-99: **SYS**
- 100-159: **Real Time**

En la estructura `proc`, cada proceso tiene:

- `p_cid`: identificador de clase
- `p_clfuncs`: punteros a funciones específicas de la clase
- `p_clproc`: datos dependientes de la clase

Clase Time Sharing (TS) en SVR4

Los procesos en esta clase tienen **prioridades dinámicas** que se recalculan al dejar la CPU:

- Si el proceso **agota** su tiempo asignado, su prioridad disminuye.

- Si el proceso se **bloquea** antes de agotarlo, su prioridad aumenta.

El quantum asignado depende de la prioridad: a mayor prioridad, menor tiempo asignado.

Campos relevantes:

- `ts_timeleft` : tiempo restante del quantum.
- `ts_cpupri` : parte del cálculo de prioridad gestionada por el sistema.
- `ts_upri` : parte controlada por el usuario mediante `priocntl()`.
- `ts_umdpr` : prioridad en modo usuario total (`ts_cpupri` + `ts_upri`).
- `ts_dispwait` : tiempo desde el inicio del quantum.

El sistema mantiene una tabla donde relaciona prioridad, quantum y reglas de actualización (`tqexp` , `slpret` , etc.).

Clase Real Time

- Prioridades fijas entre **100 y 159**.
- Quants fijos para cada nivel de prioridad.
- No se recalculan automáticamente, sólo modificables mediante `priocntl()`.

Si un proceso real time está listo pero el proceso actual está en modo kernel, la preempción se retrasa hasta que el sistema:

- Vuelva al modo usuario.
- Se bloquea.
- Llega a un **punto de preempción** (en código kernel seguro para el cambio).

Los sistemas modernos como **Solaris** eliminan estos puntos de preempción mediante el uso extensivo de semáforos, permitiendo preempción total incluso en modo kernel.

Clase system

- Reservada para procesos especiales del sistema (`pageout` , `sched` , `fsflush` ...)
- No disponible para todos los usuarios o instalaciones.
- Usa prioridades fijas entre 60 y 99.

Planificación en Linux

Linux distingue entre dos tipos de procesos:

1. Procesos de tiempo real:

- Prioridades fijas entre 1 y 99
- Algoritmos: **FIFO** y **Round-Robin**.
- Siempre se ejecuta el de mayor prioridad listo.

2. Procesos normales:

- Prioridad estática 0.
- Usan un algoritmo de prioridad dinámica con preempción.
- Sólo se ejecutan si no hay procesos de tiempo real listos.

Procesos en Unix: Ejecución en modo kernel

¿Qué significa ejecutar en modo kernel?

En los sistemas Unix, un proceso puede ejecutarse en dos modos: **modo usuario** y **modo kernel**. El **modo kernel** es un estado privilegiado en el que el sistema operativo tiene acceso completo al hardware en circunstancias específicas, ya que implica que el control pasa al núcleo del sistema operativo.

¿Cuándo se entra en modo núcleo?

Hay **tres situaciones** principales que provocan que un proceso cambie de modo usuario a modo núcleo:

1. Interrupción de dispositivo

Es un evento **asíncrono** que ocurre independientemente del proceso que esté corriendo. Un dispositivo externo necesita atención del sistema operativo. Esta interrupción puede suceder en cualquier momento: mientras se ejecuta código en modo usuario, en modo kernel o incluso durante otra rutina de interrupción

2. Excepción

Se trata de un evento **síncrono**, causado por el propio proceso en ejecución. Ejemplos comunes incluyen divisiones por cero, direcciones inválidas o instrucciones ilegales.

3. Llamadas al sistema

También es **síncrona** y ocurre cuando un proceso necesita explícitamente un servicio del sistema operativo, como leer un archivo, crear un proceso, etc.

Comportamiento del núcleo ante uno de estos eventos

Cuando ocurre cualquiera de estos eventos, el núcleo del sistema operativo toma el control del procesador:

- Guarda el contexto del procesador en su **pila del kernel**.
- Ejecuta la función correspondiente al evento.
- Una vez que termina el servicio, restaura el contexto del proceso y lo devuelve a su estado anterior, incluyendo su modo de ejecución.

Interrupciones

Las **interrupciones** son especialmente delicadas porque pueden suceder en cualquier momento. Para manejarlas adecuadamente:

- A cada interrupción se le asigna un **nivel de prioridad** llamado *IPL* o Interrupt Priority Level.
- Cuando llega una interrupción, su IPL se compara con el del proceso actual.
 - Si el IPL de la interrupción es **más alto**, se ejecuta inmediatamente el **manejador de interrupción**.
 - Si es **menor o igual**, se **pospone** hasta que el IPL del sistema sea lo suficientemente bajo.
- En general, todo el código de usuario y casi todo el código del núcleo se ejecuta con el IPL mínimo.

- En sistemas Unix tradicionales, el rango del IPL va de **0 a 7**. En sistemas BSD puede llegar hasta **31**.

Llamadas al sistema

Cuando se invoca una llamada al sistema como `read()`, `open()`, `fork()`, etc., en realidad no se accede directamente al núcleo. Lo que ocurre es lo siguiente:

1. **La biblioteca del sistema** proporciona una función wrapped, por ejemplo `read()`.
2. Esta función:
 - Coloca los parámetros de la llamada en la pila de usuario.
 - Inserta un número de servicio que identifica qué operación se solicita.
 - Ejecuta una **instrucción especial** (como `trap`, `chmk`, `int`, según la arquitectura), que hace dos cosas:
 - Cambia el modo de ejecución a **modo kernel**.
 - Transfiere el control a la rutina del núcleo que gestiona llamadas al sistema, una `syscall()`.
3. En `syscall()`
 - Se copian los argumentos a una estructura del núcleo (`u_area`).
 - Se guarda el contexto del proceso en la pila del núcleo.
 - Usando el número de servicio como índice, se accede a una tabla (`sysent[]`) que asocia cada número con una función del núcleo, por ejemplo `sys_read()`.
4. La función del kernel (`sys_read()`, etc.):
 - Presta el servicio solicitado.
 - Puede llamar a otras funciones del núcleo si es necesario.
 - Coloca los valores de retorno, o errores, en los registros apropiados.
 - Restaura el contexto del proceso y regresa al modo usuario.
 - Finalmente, el control vuelve a la función de la biblioteca y luego al programa del usuario.

Acceso a recursos

El modo núcleo proporciona mecanismos para proteger los recursos compartidos del sistema:

- Cuando un proceso entra en modo kernel, **no puede ser interrumpido por otro proceso**, lo que garantiza que puede manipular datos del kernel sin crear inconsistencias.
- Si un proceso necesita un recurso y este **ya está ocupado**, el proceso debe:
 1. Marcar el recurso como **deseado**
 2. Llamar a `sleep()`, que lo pone en estado de suspensión.
 3. Esta función invoca a `switch()`, iniciando un **cambio de contexto** y otro proceso toma el control.
- Cuando el recurso se libera:
 - Se marca como no ocupado.
 - Si había procesos esperando, se marcan como listos para ejecutar mediante `wakeup()`.

- Todos estos procesos se colocan en la **cola de procesos ejecutables**.
- Sin embargo, no se garantiza que el primer proceso despertado sea el primero en obtener el recurso. Por ello, **debe comprobar de nuevo si está disponible** antes de usarlo.

Consideraciones avanzadas sobre interrupciones y concurrencia

Aunque un proceso en modo kernel no puede ser interrumpido por otro proceso, sí puede recibir **interrupciones de hardware**. Para ello:

- Si se está accediendo a datos que podrían ser utilizados por una **rutina de interrupción**, esta interrupción debe ser **desactivada temporalmente**, elevando el IPL.
- Esta elevación debe hacerse **con cuidado**:
 - Las interrupciones deben ser atendidas rápidamente.
 - El tiempo durante el cual se desactivan debe ser **mínimo**.
 - Elevar el IPL bloquea también **todas las interrupciones con prioridad inferior**.
- Si se desea que el núcleo sea **completamente preemptible**, es decir, que incluso el código del kernel pueda ser interrumpido por otros procesos, entonces se deben usar mecanismos más robustos como **semáforos** para proteger los datos del kernel.
- En sistemas multiprocesador, la complejidad aumenta, y se requieren **mecanismos de sincronización más sofisticados** para asegurar la coherencia de los datos compartidos.

Procesos en UNIX: señales

¿Qué son las señales?

En los sistemas Unix las señales son una forma de comunicación entre el kernel y los procesos. Se utilizan para **notificar la ocurrencia de eventos asíncronos**, es decir, eventos que no se producen como consecuencia directa del flujo de ejecución del programa.

Por ejemplo, si un usuario pulsa la combinación de teclas **Ctrl+c**, el kernel envía la señal **SIGINT** al proceso en primer plano para interrumpir su ejecución. Otro ejemplo: si se cae una línea de comunicación, el sistema puede enviar la señal **SIGHUP**.

No sólo el kernel puede enviar señales; también los procesos pueden enviarse señales entre ellos mediante la llamada al sistema `kill`.

Comportamiento de los procesos ante señales

Los procesos pueden reaccionar de diferentes maneras ante la recepción de una señal:

- **Terminar el proceso.**
- **Ignorar la señal**, haciendo que no ocurra nada.
- **Definir un manejador de señal**, una función que se ejecuta cuando la señal es recibida.

Cuando el kernel detecta que debe enviar una señal, lo que realmente hace es **marcar un bit** en una estructura del proceso `proc`. La reacción se ejecuta sólo cuando el proceso **retorna al modo usuario**:

- Si el proceso está en modo kernel, la señal se aplaza hasta volver al modo usuario.
- Si el proceso está **bloqueado** esperando algo:
 - Si la espera es **interrumpible**, se interrumpe la llamada al sistema, retorna -1 y deja `errno` en EINTR.

- Si la espera no es interrumpible, la señal se trata cuando el proceso regresa a modo usuario tras completar la espera.

Señales en System V Release 2

Esta versión del sistema operativo incluía **15 señales** disponibles y algunas llamadas al sistema relacionadas:

- `kill(pid_t pid, int sig)` : permite a un proceso enviar una señal a otro.
- `signal (int sig, void (*handler)(int))` : asocia una señal a un manejador. El manejador puede ser:
 - `SIG_DFL` : la acción por defecto de la señal.
 - `SIG_IGN` : la señal se ignora.
 - Una **función definida por el usuario**, que recibe como parámetro el número de señal.

El sistema mantiene un **array en el área de proceso (`u_area`)** indexado por número de señal, que contiene los manejadores correspondientes o indica si se debe ignorar o aplicar la acción por defecto.

Cuando el proceso va a volver al modo usuario y tiene señales pendientes:

1. El kernel limpia el bit correspondiente.
2. Si la señal se ignora, no pasa nada.
3. Si hay un manejador definido:
 - Se crea un nuevo contexto en la pila del proceso.
 - Se establece el PC en la dirección del manejador.
 - La señal se restablece a su acción por defecto.

Los manejadores **no son permanentes**. Si se recibe la misma señal de nuevo, ya no se llama al manejador, salvo que se reinstale.

```
#include <signal.h>
void manejador()
{
    printf("Se ha pulado control-C\n");
}
int main()
{
    signal(SIGINT, manejador);
    while(true); //bucle infinito
}
```

En este caso, al pulsar Ctrl+C una vez se ejecuta el manejador, pero si se pulsa otra vez antes de que `signal(SIGINT, manejador);` se reinstale, el programa termina.

Versión permanente

```
#include <signal.h>
void manejador()
{
    printf("Se ha pulado control-C\n");
    signal(SIGINT, manejador); //Reinstala el manejador
}
int main()
{
    signal(SIGINT, manejador);
    while(true);
}
```

System V Release 3: Señales fiables

System V R3 introduce las **señales fiables**, donde:

- Los manejadores se vuelven permanentes.
- El manejador de una señal **se ejecuta con dicha señal bloqueada**.
- Se puede **bloquear y desbloquear señales** manualmente.
- El estado de las señales se almacena en la estructura `proc`.

Nuevas llamadas al sistema

- `sigset(int señal, void (*handler)(int))` : instala un manejador permanente que no puede ser interrumpido por la misma señal.
- `sighold(int señal)` : bloquea una señal
- `sigrelse(int señal)` : desbloquea una señal
- `sigpause(int señal)` : desbloquea la señal indicada y bloquea el proceso hasta que reciba una señal.

Señales en BSD (Berkeley Software Distribution)

BSD mejora el control de señales con varias características:

- **Bloqueo de señales en grupo**, mediante máscaras de bits.
- Si una señal **interrumpe una llamada al sistema**, esta se reinicia automáticamente.

Nuevas llamadas

- `sigsetmask(int mask)` : define el conjunto de señales bloqueadas.
- `sigblock(int mask)` : añade señales al conjunto bloqueado.
- `sigpause(int mask)` : establece la máscara y suspende el proceso.
- `sigvec(int sig, struct sigvec *vec, struct sigvec *ovec)` : instala el manejador para una señal.
- `sigstack(struct sigstack *ss, struct sigstack *oss)` : define una **pila alternativa** para los manejadores.

System V Release 4: Estándar moderno

System V R4 integra y mejora todas las funcionalidades anteriores. Es la base de la mayoría de sistemas Unix actuales.

Llamadas principales

- `sigprocmask(int how, const sigset_t *set, sigset_t *oldset)` : gestiona la **máscara de señales bloqueadas** del proceso.
 - `how` puede ser:
 - `SIG_BLOCK` : añade señales al conjunto bloqueado.
 - `SIG_UNBLOCK` : elimina señales del conjunto bloqueado.
 - `SIG_SETMASK` : reemplaza el conjunto entero.
 - `oldset` permite recuperar el conjunto anterior.
- Manipulación de conjuntos de señales (`sigset_t`):

```
int sigemptyset(sigset_t *set); // Vacía el conjunto
int sigfillset(sigset_t *set); // Llena el conjunto con todas las señales
int sigaddset(sigset_t *set, int signum); // Añade una señal
int sigdelset(sigset_t *set, int signum); // Elimina una señal
int sigismember(const sigset_t *set, int signum); // Verifica si una señal
está en el conjunto
```

- `sigsuspend(const sigset_t *mask)` : establece una máscara y **suspende el proceso** hasta recibir una señal que no esté bloqueada ni ignorada.
- `sigpending(sigset_t *set)` : consulta si hay señales pendientes.
- `sigaltstack(const stack_t **ss, stack_t *oss)` : define una pila alternativa para los manejadores.
- `sigsendset(procset_t *psp, int sig)` y `sigsend(idtype_t idtype, id_t id, int sig)` : permiten enviar señales con más opciones que kill.

sigaction : Control completo del comportamiento de las señales

Una de las llamadas más potentes es `sigaction` , que permite definir de forma muy precisa cómo debe reaccionar un proceso a una señal.

```
int sigaction(int signum, const struct sigaction *act, struct sigaction
*oldact);
```

La estructura de `sigaction` contiene:

- `sa_handler` : puede ser `SIG_DFL` , `SIG_IGN` o la dirección de un manejador.
- `sa_mask` : conjunto de señales a bloquear durante la ejecución del manejador.
- `sa_flags` : modificadores del comportamiento, entre ellos:
 - `SA_ONSTACK` : el manejador se ejecuta en la pila alternativa.
 - `SA_RESETHAND` : el manejador es no permanente.
 - `SA_NODEFER` : la señal no se bloquea durante la ejecución del manejador.
 - `SA_RESTART` : si una señal interrumpe una llamada al sistema, esta se reinicia automáticamente.
- Otros: `SA_SIGINFO` , `SA_NOCLDWAIT` , `SA_NOCLDSTOP` , `SA_WAITSIG` .

Comunicación entre procesos

En los sistemas Unix, existen diversos mecanismos para que los procesos puedan comunicarse entre sí. Estos mecanismos forman parte fundamental del sistema operativo y permiten que los procesos intercambien información o se coordinen en sus acciones. Los principales métodos de comunicación entre procesos (Inter Process Communication) son:

- Tuberías o pipes.
- Memoria compartida.
- Semáforos.
- Colas de mensajes.

Pipes

Los pipes son un método sencillo y eficiente para permitir la comunicación entre procesos. Funcionan como archivos temporales que pueden ser utilizados para enviar información de un proceso a otro.

Para crear una tubería se utiliza la llamada al sistema `pipe()` :

```
#include <unistd.h>
int pipe(int fildes[2]);
```

Esta función devuelve dos descriptores de archivo:

- `fildes[0]` : extremo de **lectura**
- `fildes[1]` : extremo de **escritura**

En algunos sistemas, ambos extremos pueden ser usados tanto para lectura como para escritura, pero en la mayoría se sigue usando el estándar histórico, se utilizan de forma esperada: uno exclusivamente para leer y otro para escribir.

Funcionamiento

- Cuando **no hay datos** en la tubería, una llamada a `read()` se bloquea hasta que haya algo disponible.
- Cuando la tubería está **llena**, una llamada `write()` se bloquea hasta que haya espacio.
- Los datos se **descartan automáticamente** una vez han sido leídos.

Memoria compartida

La memoria compartida permite que varios procesos accedan directamente a una región común de memoria. Este mecanismo es más rápido que otros métodos, ya que evita la copia de datos entre procesos.

Paso 1: Obtener un bloque de memoria

Para usar memoria compartida, primero se debe obtener, o crear, un segmento de memoria con

`shmget()` :

```
#include <sys/types.h>
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
int shmget(key_t key, size_t size, int shmflg);
```

- `key` : clave numérica que identifica la región.
- `size` : tamaño de la región de memoria.
- `shmflg` : permisos y banderas.

Uso de las banderas:

- 0: devuelve el identificador si existe; error si no.
- `IPC_CREAT` : crea el recurso si no existe.
- `IPC_CREAT | IPC_EXCL` : error si ya existe, útil para evitar reutilizar recursos por error.

Paso 2: Asociar la memoria al proceso

Una vez creado el segmento, se debe "adjuntar" al espacio de direcciones de memoria con

`shmat()` :

```
void *shmat(int shmid, const void *shmaddr, int flg);
```

- `shmid` : identificador del segmento, devuelto por `shmget` .
- `shmaddr` : dirección virtual, usar NULL para que el sistema lo asigne.
- `flg` : opciones como `IPC_RDONLY`

Devuelve un puntero a la región de memoria compartida que el proceso puede usar para acceder directamente a los datos.

Paso 3: Desasociar la memoria.

Cuando ya no se necesita, el segmento debe ser "desmontado" con `int shmdt(char *shmaddr);`

Paso 4: Control del recurso

Para operaciones avanzadas como eliminar la memoria del sistema o consultar información, se

usa `shmctl()` : `int shmctl(int shmid, int cmd, struct shmid_ds *buf);`

Comandos como `IPC_RMID` (eliminar), `IPC_STAT` , `IPC_SET` , permiten administrar el recurso.

```
void *ObtenerMemoria(key_t clave, off_t tam, int crear)
{
    int id;
    void *p;
    int flags = 0666;
    if(crear)
        flags |= IPC_CREAT | IPC_EXCL;
    if((id = shmget(clave, tam, flags)) == -1)
        return NULL;
    if((p = shmat(id, NULL, 0)) == (void *) -1)
    {
        if(crear)
            shmctl(id, IPC_RMID, NULL);
        return NULL;
    }
}
```

```
    return p;
}
```

Semáforos

Los semáforos permiten sincronizar el acceso a recursos compartidos entre múltiples procesos. Unix proporciona una interfaz para manejar **arrays de semáforos**, especialmente útil cuando se usan junto a memoria compartida.

Paso 1: crear o acceder a un array de semáforos

```
int semget(key_t key, int nsems, int semflg);
```

- `key` : clave del recurso.
- `nsems` : número de semáforos en el array.
- `semflg` : permisos y banderas como en `shmget` .

Paso 2: Realizar operaciones sobre semáforos

```
int semop(int semid, struct sembuf *sops, size_t nsops);
```

Se pueden realizar varias operaciones con un array de estructuras `sembuf` _

```
struct sembuf{
    ushort_t sem_num; // número de semáforo
    short sem_op;      // operación a realizar
    short sem_flg;     // flags (SEM_UNDO, IPC_NOWAIT)
};
```

`sem_op` puede ser:

- negativo: espera a que el valor sea suficiente y lo decrementa.
- cero: espera a que el valor sea 0.
- positivo: incrementa el valor del semáforo.

Paso 3: Control del recurso

```
int semctl(int semid, int semnum, int cmd, ...);
```

Permite eliminar, consultar o modificar semáforos.

El cuarto argumento puede ser un union `semun`, con distintas posibilidades:

```
union semun{
    int val;
    struct semid_ds *buf;
    ushort *array;
};
```

Colas de mensajes

Las colas de mensajes permiten el envío de bloques de datos estructurales entre procesos. A diferencia de los pipes, aquí los mensajes pueden tener un **tipo**, lo que permite una mayor flexibilidad.

Paso 1: Crear o acceder a una cola de mensajes

```
int msgget(key_t key, int msgflg);
```

Como en los recursos anteriores, se usa una clave y unas banderas.

Paso 2: Enviar y recibir mensajes

```
int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);
```

```
int msgrcv(int msqid, void *msgp, size_t msgsz, long msgtyp, int msgflg);
```

- `msgsz` : tamaño del mensaje sin incluir el campo de tipo.
- `msgtyp` : tipo de mensaje a recibir:
 - 0: el primero de la cola.
 - n: el primero con tipo n.
 - -n: el primero con tipo menor o igual a n.
- `msgflg` : permite operaciones como `IPC_NOWAIT` , `MSG_NOERROR` .

Paso 3: Control de la cola

```
int msgctl(int msqid, int cmd, struct msqid_ds *buf);
```

Con `msgctl()` se puede eliminar una cola (`IPC_RMID`) o consultar/modificar su estado (`IPC_STAT` , `IPC_SET`).

Concurrencia

¿Qué es un spooler?

El término *spool* proviene de "Simultaneous Peripheral Operations On-Line", y se refiere a una técnica que permite gestionar múltiples trabajos entrada/salida de formas simultánea, almacenándolos temporalmente en una cola antes de ser procesados. En el caso de las impresoras, un *spooler* permite que varios procesos puedan enviar documentos a imprimir sin que se interfieran directamente entre ellos. El demonio de impresión (printer daemon) consulta el *spooler* y se encarga de imprimir los archivos en el orden adecuado. Imaginemos una estructura donde el *spooler* se comporta como un buffer. En este contexto, tenemos dos variables compartidas importantes:

- `in` : indica la proxima posición libre del *spooler*.
- `out` : indica la siguiente posición que el demonio de impresión debe imprimir.

Condiciones de carrera

Este fenómeno se conoce como **condición de carrera**, y aparece cuando varios procesos acceden a recursos compartidos sin una coordinación adecuada. El resultado del programa depende entonces del orden en que se intercalan las instrucciones de los procesos, lo cual introduce comportamientos inesperados y difíciles de depurar, ya que los errores no siempre se manifiestan.

Sección crítica y su control

¿Cómo evitamos las condiciones de carrera?

El primer paso es identificar las **secciones críticas**, es decir, aquellas partes del código donde se accede o modifica un recurso compartido. Si varios procesos ejecutan estas secciones de

forma simultánea, pueden producirse errores como el descrito anteriormente.

Para evitarlo, cualquier solución al problema de la sección crítica debe garantizar:

- **Exclusión mutua:** sólo un proceso puede estar ejecutando su sección crítica en un momento dado.
- **Progreso:** si ningún proceso está en su sección crítica, no debe impedirse que otros procesos accedan a ella.
- **Espera limitada:** ningún proceso debe esperar indefinidamente para acceder a su sección crítica.
- **Independencia del número o velocidad de los procesadores:** la solución debe funcionar correctamente sin depender del entorno hardware.

Atomicidad

Una herramienta clave para implementar soluciones a estos problemas es la **atomicidad**. Una secuencia de instrucciones se considera **atómica** si el sistema operativo garantiza que no habrá cambios de contexto durante su ejecución. Es decir, se ejecuta de manera indivisible. Gracias a esto, se han propuesto varias soluciones clásicas al problema de la sección crítica, como:

- **semáforos:** propuestos por Dijkstra en 1965
- **monitores:** introducidos por Hansen en 1973 y Hoare en 1974

La solución más común en sistemas UNIX es el uso de **semáforos**.

Semáforos

Un **semáforo** es una variable entera que sólo puede ser manipulada a través de dos operaciones atómicas:

1. P(sem) o Wait(sem): `wait` hasta que `sem > 0`; luego `sem--`;
Si el valor del semáforo es cero, el proceso queda **bloqueado**, sin consumir CPU.
2. V(sem) o Signal(sem): `sem++`;

Si hay procesos esperando en el semáforo, uno de ellos es **despertado** por el sistema operativo, que se encarga de reanudarlo de forma atómica.

Un **semáforo binario** es un caso particular donde el semáforo puede tener los valores 0 o 1, lo cual lo convierte en una herramienta ideal para garantizar exclusión mutua.

Solución al problema del spooler con semáforo

Podemos resolver el problema original del *spooler* utilizando un semáforo binario mutex, con valor inicial igual 1. Esto asegura que sólo un proceso pueda entrar en su sección crítica en un momento dado:

```
void printFile(char *fname)
{
    wait(mutex); // Entrada sección crítica
    load shared var "in" into reg;
    spooler[reg] = fname;
    write reg + 1 in shared var "in"
```

```
signal(mutex); // Salida sección crítica  
}
```

De igual manera, el demonio de impresión, que accede al *spooler* para imprimir archivos, también debe proteger su acceso usando el mismo semáforo mutex.

Entrada / Salida

Entrada y salida

Los dispositivos de entrada/salida permiten que la CPU se comuniquen con el mundo exterior. Ejemplos comunes de estos dispositivos incluyen teclados, pantallas, impresoras y discos.

La comunicación entre la CPU y un elemento externo es similar a la comunicación con la memoria principal, ya que en ambos casos se pueden leer y escribir datos. No obstante, el comportamiento de los dispositivos de entrada/salida es distinto. Por ejemplo, los datos no siempre están disponibles.

Debido a estas diferencias de comportamiento, los métodos de acceso utilizados para los dispositivos de entrada/salida también son distintos a los utilizados para acceder a la memoria.

Estructura de un sistema de entrada/salida

En teoría, los dispositivos de entrada/salida podrían comunicarse con la CPU a través de los buses del sistema. Sin embargo, debido a la gran heterogeneidad de estos dispositivos, sería muy costoso que la CPU los gestionara directamente. El sistema operativo tendría que conocer todos los detalles de cada dispositivo, y además, la CPU es mucho más rápida que la mayoría de los dispositivos de entrada/salida.

Por ello, los dispositivos están conectados a un componente hardware intermedio denominado **controlador de dispositivos**.

El controlador de dispositivo recibe comandos desde la CPU y se encarga de transmitirlos al dispositivo correspondiente. Por ejemplo, puede recibir la orden "escribir el bloque 2534 en el disco" y traducirla a las instrucciones físicas necesarias para realizar esa operación. De esta forma, la CPU queda liberada de tareas de muy bajo nivel, como especificar el lado, cilindro o sector del disco.

El controlador actúa, por tanto, como una interfaz entre la CPU y el dispositivo de entrada / salida. Puede gestionar uno o varios dispositivos del mismo tipo. Por ejemplo, un controlador IDE puede controlar dos discos duros, o bien un disco duro y una unidad DVD.

Comunicación entre controlador y CPU

Los controladores se comunican con la CPU mediante registros o puertos específicos, que suelen incluir:

- **Registros de control:** permite enviar comandos al dispositivo.

- **Registros de estado:** proporciona información sobre el estado del dispositivo o del propio controlador. Por ejemplo:
 - Si el comando anterior ya se ha completado.
 - Si se han producido errores en la última operación de entrada/salida.
- **Registros de datos:** pueden ser de entrada, de salida, o bidireccionales. Normalmente, son registros capaces de almacenar entre 1 y 8 bytes.

Algunos controladores integran chips FIFO que permiten almacenar temporalmente pequeñas cantidades de datos en ráfagas, hasta que la CPU o el dispositivo estén listos para recibirlos.

Funciones del controlador de dispositivo

El controlador es responsable de varias tareas fundamentales:

- **Coordinación de flujo de datos** entre la CPU y el dispositivo periférico.
- **Comunicación con el dispositivo de entrada/salida**, que incluye:
 - Envío de comandos.
 - Intercambio de datos.
 - Recepción de información sobre el estado del dispositivo.
- **Almacenamiento temporal de datos** (buffer), necesario debido a la gran diferencia de velocidad entre la CPU y el dispositivo. Por ejemplo, cuando la CPU envía datos más rápido de lo que el dispositivo puede procesarlos, el controlador actúa como intermediario, reteniéndolos y enviándolos de forma escalonada.
- **Detección de errores** durante la operación de entrada/salida

Estructura del software de entrada/salida

Abstracción y encapsulamiento

Un programa puede escribir en un disco duro sin conocer el tipo específico de disco gracias a la **abstracción**. Este principio permite eliminar las diferencias entre dispositivos al identificarlos dentro de unas pocas clases o tipos genéricos. Para cada clase se define una interfaz estándar con un conjunto fijo de funciones.

La **encapsulación** complementa este proceso: las diferencias reales entre dispositivos se ocultan dentro de los *device drivers*. Estos controladores se adaptan internamente a las particularidades de cada dispositivo, pero ofrecen hacia el exterior una interfaz estándar. Así, los programas pueden interactuar con los dispositivos sin conocer sus detalles internos.

Capas del software de E/S

El software de entrada/salida está estructurado en capas. Esta organización facilita tanto la incorporación de nuevos dispositivos como el desarrollo del sistema operativo.

- **Capa del controlador de dispositivos:** oculta las diferencias entre los distintos *device drivers* frente al núcleo del sistema. Gracias a esto, las llamadas al subsistema de E/S pueden tratar a los dispositivos como si perteneciesen a unas pocas clases genéricas.
- **Subsistema de E/S del sistema operativo:** se sitúa por encima de la capa de controladores, y permite que tanto los desarrolladores del sistema operativo como los

fabricantes de hardware trabajen de forma más eficiente. Los fabricantes pueden o bien ajustar su hardware a interfaces existentes, o bien escribir el controlador de dispositivo adecuado para su integración.

Capas específicas del software de E/S

El software de entrada/salida se divide en las siguientes capas funcionales:

- **Software a nivel de usuario:** proporciona la interfaz entre el usuario y el sistema. Incluye tareas como el formateo de datos.
- **Software independiente del dispositivo:** se encarga de tareas como la asignación de espacio, el control de privilegios o el uso de caché, en caso de existir. No depende del dispositivo físico concreto.
- **Controlador de dispositivo:** es la única capa que se comunica directamente con el controlador del dispositivo. Toda la lógica específica del dispositivo está contenida en esta capa. Para que un sistema operativo pueda utilizar un nuevo dispositivo, basta con incorporar su correspondiente *driver*.
- **Gestión de interrupciones:** se encarga de gestionar las interrupciones que provienen de los dispositivos, despertando procesos que esperaban una operación de E/S o liberando recursos.

Tipos de entrada/salida

Según la forma en que se realiza la comunicación entre la CPU y los dispositivos, y según la percepción que tiene el proceso respecto al desarrollo de la operación de entrada/salida, podemos distinguir varios tipos de E/S.

Según el método de comunicación entre la CPU y los dispositivos

Entrada/Salida mapeada en memoria

En este modelo, los dispositivos aparecen dentro del espacio de direcciones. Es decir, la memoria y los dispositivos comparten el mismo espacio de direcciones.

Esto no suele suponer un problema, ya que el espacio necesario para E/S suele ser mucho menor que el espacio total de direcciones disponible. Para acceder a los registros del controlador del dispositivo se utilizan instrucciones del tipo MOV, como en cualquier operación normal de lectura o escritura en memoria.

Entrada/salida con espacio de puertos separado

En este caso, se dispone de un conjunto de direcciones explícitas dedicadas exclusivamente a E/S. A cada registro de control de un dispositivo se le asigna un número de puerto E/S, y el conjunto de todos ellos forma lo que se conoce como *espacio de entrada/salida*.

Normalmente, se emplean instrucciones especiales para acceder a este espacio, como IN o OUT en la arquitectura Intel. En otros sistemas, como PowerPC, un registro de control puede activar el uso de este espacio.

Según la percepción del proceso sobre la operación de E/S

Entrada/Salida Sincrónica

En este caso, el proceso percibe que debe esperar hasta que la operación de E/S se complete. Desde su punto de vista, no puede continuar su ejecución hasta que haya finalizado la entrada o salida.

Entrada/Salida Asíncrona

Aquí, el proceso percibe que no necesita esperar a que la operación de E/S termine. Puede seguir ejecutándose y, en algún momento, será notificado de que la operación ha finalizado.

Dado que la CPU es mucho más rápida que los dispositivos de E/S, el sistema operativo cede el uso de la CPU a otra tarea mientras la operación de entrada/salida se lleva a cabo. El proceso que inició la operación pasa a estado de *espera*. Aunque el proceso "cree" que la operación es sincrónica, en realidad el sistema la está manejando de forma asíncrona. Cuando la E/S concluye, el proceso pasa de la cola de I/O a la cola de listos.

Además, muchos sistemas operativos permiten realizar operaciones de entrada/salida verdaderamente asíncronas. En este caso, el proceso inicia la operación y continúa su ejecución sin interrupciones. Más adelante, será notificado por el sistema operativo cuando la operación haya concluido.

Esta notificación puede realizarse, por ejemplo:

- Mediante una interrupción.
- Modificando el valor de una variable en el espacio de usuario del proceso.

Métodos para realizar operaciones Entrada/Salida

Existen distintos métodos para llevar a cabo operaciones de entrada/salida entre la CPU y los dispositivos. A continuación se describen los principales enfoques utilizados: mediante sondeo (polling), mediante interrupciones, y mediante acceso directo a memoria.

Entrada/Salida con sondeo

Es el método más simple para realizar una operación de entrada/salida.

Para lograr la sincronización entre el dispositivo y la CPU, esta última debe consultar repetidamente al dispositivo para comprobar si tienen un nuevo dato disponible para ser leído o si está listo para recibir un nuevo dato.

Este enfoque genera lo que se conoce como *espera activa*, en la que se desperdicia tiempo de CPU cada vez que se realiza una consulta al dispositivo. Cuanto más frecuente sea esta consulta, más tiempo se pierde.

El sondeo resulta lento y, además, puede dar lugar a la pérdida de datos si el dispositivo no es consultado con la suficiente frecuencia.

Entrada/Salida controlada por interrupciones

En este método, es el propio dispositivo el que solicita la atención de la CPU mediante una interrupción. Cuando se detecta una interrupción, el sistema operativo:

1. Detiene el proceso que estaba en ejecución y guarda su estado.
2. Transfiere el control al *interrupt handler*
3. Ejecuta dicho procedimiento.

4. Una vez finalizado, continúa la ejecución del proceso detenido desde el punto en el que se encontraba.

Estos procedimientos de servicio están ubicados en posiciones de memoria indicadas por los vectores de interrupción. Durante el proceso de inicialización, el sistema operativo se encarga de comprobar qué dispositivos están conectados e instala los correspondientes *drivers* en el vector de interrupciones.

El vector de interrupciones se encarga de dirigir cada interrupción al manejador adecuado. Una vez finalizado el servicio de interrupción, normalmente se invoca al planificador de tareas, y el dispositivo, por ejemplo una impresora, vuelve a generar una interrupción cuando está listo para continuar con la transferencia del siguiente dato.

Entrada/Salida mediante acceso directo a memoria

Tanto en la entrada/salida con sondeo como con interrupciones, la CPU es quien dirige la transferencia de datos cuando estos están listos, bien sea consultando el registro de estado o recibiendo una interrupción del dispositivo.

Esto presenta algunos inconvenientes:

- La velocidad de transferencia está limitada por la velocidad de la CPU al mover los datos entre los registros/memoria y el controlador o dispositivo.
- Mientras se realiza la transferencia, la CPU debe encargarse de dirigir dicha operación, impidiéndole atender otras tareas.

Aunque estos problemas son menores en dispositivos lentos que transfieren poca cantidad de datos, se vuelven críticos en dispositivos que manejan grandes volúmenes de información, como los discos duros.

Para resolverlo, se utiliza un controlador de DMA, que libera a la CPU de la gestión directa de la transferencia de datos. En este caso, la CPU sólo interviene al final del proceso, cuando recibe una interrupción que le notifica la finalización de la transferencia.

El controlador DMA se encarga por completo de transferir los datos al dispositivo. Genera todas las señales de dirección y control del bus necesarias. La CPU debe proporcionar al DMA tres elementos:

- El tipo de operación,
- La dirección de memoria involucrada en la transferencia,
- El número de bytes a transferir.

Cuando la transferencia concluye, el DMA genera una interrupción para avisar a la CPU. Como tanto el DMA como la CPU deben utilizar el mismo bus, existen distintos métodos para compartirlo:

Modos de compartir el bus

1. DMA en modo ráfaga

Una vez que el DMA toma el control del bus, transfiere un bloque completo de datos. La CPU debe esperar hasta que finalice toda la transferencia.

Durante este tiempo, la CPU puede seguir ejecutando instrucciones y accediendo a datos desde la caché. Este es el método más rápido.

Se utiliza principalmente en dispositivos de almacenamiento secundario como discos, ya que al ser costoso obtener acceso al bus, se aprovecha para transferir varios datos de una vez.

2. DMA con robo de ciclos

El DMA transfiere una sola palabra cada vez que toma el control del bus, y luego cede el control nuevamente a la CPU.

La transferencia de datos se realiza mediante una secuencia de ciclos DMA intercalados con ciclos de CPU.

3. Bus transparente

El DMA sólo usa el bus cuando la CPU no lo está utilizando, por ejemplo, mientras la CPU está decodificando una instrucción o utilizando la ALU.

Durante la transferencia, la CPU puede seguir accediendo a datos e instrucciones que se encuentren en la caché.

Estructura de discos

Un disco tradicional está formado por un conjunto de platos que giran de manera conjunta.

Cada una de las superficies de los platos se denominan cara o lado, y cada una de ellas contiene una serie de pistas circulares concéntricas.

El conjunto de pistas que se sitúan en la misma posición de los distintos lados conforma lo que se llama un cilindro. Cada cilindro, a su vez, está dividido en sectores, que constituyen la unidad básica de entrada/salida en un disco duro. Típicamente, cada sector tiene una capacidad de 512 bytes.

Para referirse de forma precisa a un sector, se utilizan tres coordenadas: el cilindro, el lado y el número de sector. Por convención, la numeración de cilindros y lados comienza desde 0, mientras que los sectores suelen numerarse a partir de 1.

El sistema operativo no accede directamente a las características del disco. En su lugar, lo visualiza como una secuencia de bloques lógicos. Por ejemplo, un disquete de 1,4 MB tiene 80 cilindros, 2 lados y 18 sectores por pista. Si este disco se formatea con bloques de 4 KB, el sistema operativo lo tratará como una secuencia de 360 bloques lógicos, cada uno de ellos formado por 8 sectores físicos.

La asignación del espacio se realiza a nivel de bloques lógicos, no de sectores físicos.

Particiones de disco

Los discos de gran tamaño pueden dividirse en una o más áreas, conocidas como particiones. Cada partición puede utilizarse como un sistema de archivos independiente.

De forma tradicional, el primer sector del disco contiene una tabla que describe las distintas particiones del disco. El formato de esta tabla varía según el sistema operativo:

- En sistemas Windows y Linux, se utiliza una tabla de particiones de tipo MBR, que permite definir hasta 4 entradas.
- En sistemas con particiones de tipo GPT, se pueden definir hasta 128 entradas, aunque la tabla de particiones ocupa más de 512 bytes.
- En sistemas BSD se emplea una estructura denominada *disklabel*, que puede contener 8 o 16 entradas.

Planificación de discos

Cuando en un sistema se generan múltiples particiones de entrada/salida dirigidas al disco, estas pueden organizarse mediante algoritmos de planificación. La planificación de discos es responsabilidad del sistema operativo, cuyo objetivo es usar el hardware de forma eficiente. En el caso de los discos, esto implica minimizar el tiempo de acceso.

El tiempo de acceso a un bloque de disco tiene dos componentes principales:

- **Tiempo de búsqueda:** es el tiempo necesario para mover las cabezas del disco hasta el cilindro que contiene el bloque deseado.
- **Latencia rotacional:** es el tiempo que se espera a que el disco gire y posicione el sector deseado bajo la cabeza de lectura/escritura.

Otros tiempos, como el de transferencia de bloque, suelen considerarse constantes. Dado que el tiempo de búsqueda suele ser el factor dominante, la planificación de discos se enfoca habitualmente en reducirlo.

Aunque también existen técnicas para reducir la latencia rotacional, en sistemas modernos el controlador suele leer pistas completas y luego transfiere únicamente los sectores requeridos.

Algoritmos de planificación de disco

Existen varios algoritmos para decidir el orden en que se atienden las peticiones de disco:

- **First Come First Served:** Las peticiones se atienden en orden de llegada.
- **Shortest Seek Time First:** Se atienden primero las peticiones que requieran menor desplazamiento de la cabeza desde su posición actual. Aunque reduce el tiempo medio de búsqueda, puede provocar inanición de algunas peticiones alejadas.
- **SCAN:** La cabeza del disco se desplaza en una dirección, atendiendo todas las peticiones en su camino. Al llegar al extremo, invierte su dirección y continúa sirviendo las peticiones pendientes en el nuevo sentido.
- **C-SCAN:** Similar a SCAN, pero cuando la cabeza alcanza el final del disco, vuelve directamente al inicio sin atender ninguna petición en el trayecto de regreso. Esto proporciona un tiempo de espera más uniforme para cada cilindro.
- **SCAN /C-SCAN con N pasos:** Son variantes de los anteriores, pero en cada recorrido la cabeza atiende como máximo N peticiones.

- **C-LOOK**: Variante de C-SCAN. En lugar de llegar hasta el último cilindro, la cabeza se detiene en el último cilindro que tiene una petición pendiente. De igual forma, al cambiar de dirección no necesariamente vuelve al cilindro 0, sino al primero con petición pendiente.

Otros algoritmos de planificación

- **Complete Fair Queuing**: Este algoritmo asigna distintas colas a las peticiones de disco provenientes de distintos procesos. A cada cola se le otorga un quantum de tiempo, cuya duración y número de peticiones que pueden procesarse dependen de la prioridad de entrada/salida (i/o) del proceso. En Linux, el comando `ionice` permite modificar dicha prioridad.
- Existen otros algoritmos en Linux, como *noop*, *deadline* o *anticipatory scheduling*, que ofrecen distintas estrategias según las necesidades del sistema.

Entrada/Salida en UNIX

Dispositivos de entrada/salida en UNIX

En Unix, los dispositivos aparecen representados como archivos, habitualmente ubicados en el directorio `/dev`.

En algunos sistemas, como Solaris, estos archivos pueden ser enlaces simbólicos que apuntan al archivo real del dispositivo.

Cada dispositivo se asocia a un inodo. Este inodo indica si se trata de un dispositivo de bloque o de carácter, y contiene dos números: el número mayor (major) y el número menor (minor). El número mayor identifica el manejador de dispositivo (driver) que se debe utilizar para acceder al dispositivo, mientras que el número menor especifica cuál unidad concreta, de las gestionadas por ese manejador, se está utilizando.

Podemos acceder a los dispositivos mediante las mismas llamadas del sistema que se usan para acceder a archivos (`open`, `read`, `write`), siempre y cuando el proceso tenga los privilegios adecuados. Para acceder a características o funcionalidades adicionales del dispositivo, se utiliza la llamada `ioctl`.

Llamadas de E/S en UNIX

En UNIX se usa E/S no formateada: se leen o escriben secuencias de bytes. Las llamadas de E/S reciben un descriptor de archivo, una dirección de memoria y el número de bytes a transferir.

```
int open(const char *path, int flags, mode_t mode);
```

Recibe la ruta del archivo (`path`), un modo de apertura, como `O_RDONLY`, `O_WRONLY`, `O_RDWR`, y en caso de creación, los permisos. Devuelve un descriptor de archivo, que se usará en las siguientes llamadas a `read` o `write`.

```
int close(int fd);
```

Cierra el descriptor de archivo `fd`, previamente obtenido mediante `open`.

```
ssize_t read(int fd, void *buf, size_t count);
```

```
ssize_t write(int fd, const void *buf, size_t count);
```

Estas llamadas reciben un descriptor de archivo, una dirección de memoria y el número de bytes a transferir. Devuelven el número de bytes efectivamente leídos o escritos. La operación se realiza desde la posición actual del archivo, que se actualiza tras cada llamada.

```
off_t lseek(int fd, off_t offset, int whence);
```

Permite modificar la posición actual dentro del archivo. El desplazamiento se interpreta según el valor de whence:

- `SEEK_SET` : desde el principio del archivo.
- `SEEK_CUR` : desde la posición actual.
- `SEEK_END` : desde el final del archivo.

```
ssize_t pread(int fd, void *buf, size_t count, off_t offset);
```

```
ssize_t pwrite(int fd, const void *buf, size_t count, off_t offset);
```

Funcionan como read y write, pero con un parámetro adicional que indica el desplazamiento desde el inicio del archivo. Estas llamadas **no** modifican la posición actual.

```
ssize_t readv(int fd, const struct iovec *iov, int iovcnt);
```

```
ssize_t writev(int fd, const struct iovec *iov, int iovcnt);
```

```
ssize_t preadv(int fd, const struct iovec *iov, int iovcnt, off_t offset);
```

```
ssize_t pwritev(int fd, const struct iovec *iov, int iovcnt, off_t offset);
```

Estas llamadas permiten realizar operaciones de E/S sobre varias regiones de memoria no contiguas. Cada struct iovec describe un búfer: su dirección (iov_base) y su tamaño en bytes (iov_len).

Entrada/Salida asíncrona

En Unix, los procesos pueden realizar E/S de manera asíncrona, es decir, iniciar una operación de entrada/salida y recibir una notificación cuando esta se completa.

Las funciones que permiten este tipo de operación pertenecen al estándar POSIX de tiempo real y deben entrelazarse con `-lrt`.

Funciones principales

- `int aio_read(struct aiocb *aiocbp);`
- `int aio_write(struct aiocb *aiocbp);`
- `ssize_t aio_return(struct aiocb *aiocbp);`
- `int aio_error(struct aiocb *aiocbp);`
- `int aio_cancel(struct aiocb *aiocbp);`

```
struct aiocb {
    int aio_fildes;           // Descriptor de archivo
    off_t aio_offset;         // Desplazamiento dentro del archivo
    volatile void *aio_buff;  // Dirección del búfer
    size_t aio_nbytes;        // Número de bytes a transferir
    int aio_reqprio;          // Prioridad de la petición
    struct sigevent aio_sigevent; // Método de notificación
    int aio_lio_opcode;        // Operación (sólo para lio_listio)
    ...
}
```

Notificación

El campo `aio_sigevent` define cómo debe notificarse al proceso cuando la operación finaliza. El tipo de notificación puede tomar valores como:

- `SIGEV_NONE`
- `SIGEV_SIGNAL`
- `SIGEV_THREAD`

Otros campos relevantes son `sigev_signo`, `sigev_value`, `sigev_notify_function`, `sigev_notify_attributes` y `sigev_notify_thread_id`.

Redirección de entrada, salida y error

Los descriptores de archivo 0, 1 y 2 corresponden respectivamente a la entrada estándar (`STDIN_FILENO`), salida estándar (`STDOUT_FILENO`) y error estándar (`STDERR_FILENO`) de un proceso. Estos flujos pueden redirigirse a archivos usando la shell. Por ejemplo, en Bash se utilizan los símbolos:

- `<` para redirigir la entrada.
- `>` para redirigir la salida.
- `&>` para redirigir el error.

Redirección mediante llamadas del sistema

Para hacer redirección de flujos desde código, pueden utilizarse las llamadas `dup`, `dup2` y `fcntl` con opción `F_DUPFD`.

Uso de dup: `int dup(int fd);`

Esta función duplica un descriptor de archivo, asignando el número más bajo disponible. El descriptor nuevo y el original se refieren al mismo archivo abierto, compartiendo sus indicadores y la posición actual (`cur_pos`). Por tanto, una llamada a `lseek` en uno de ellos afectará también al otro.